

第10章　控制单元的设计

计算机组成原理

大　　纲

(一) CPU的功能和基本结构

(二) 指令执行过程

(三) 数据通路的功能和基本结构

(四) 控制器的功能和工作原理

1、硬布线控制器

2、微程序控制器微程序、微指令和微命令 ;微指令的编码方式; 微地址的形式方式。

(五) 异常和中断机制

1. 异常和中断的基本概念
2. 异常和中断的分类
3. 异常和中断的检测与响应

(六) 指令流水线

1、指令流水线的基本概念

2、指令流水线的基本实现

3、结构冒险、数据冒险和控制冒险的处理

4.超标量和动态流水线的基本概念

(七) 多处理器基本概念

1、SISD、SIMD、MIMD、向量处理器的基本概念

2、硬件多线程的基本概念

3、多核处理器(multi-core)的基本概念

4、共享内存多处理器(SMP)的基本概念

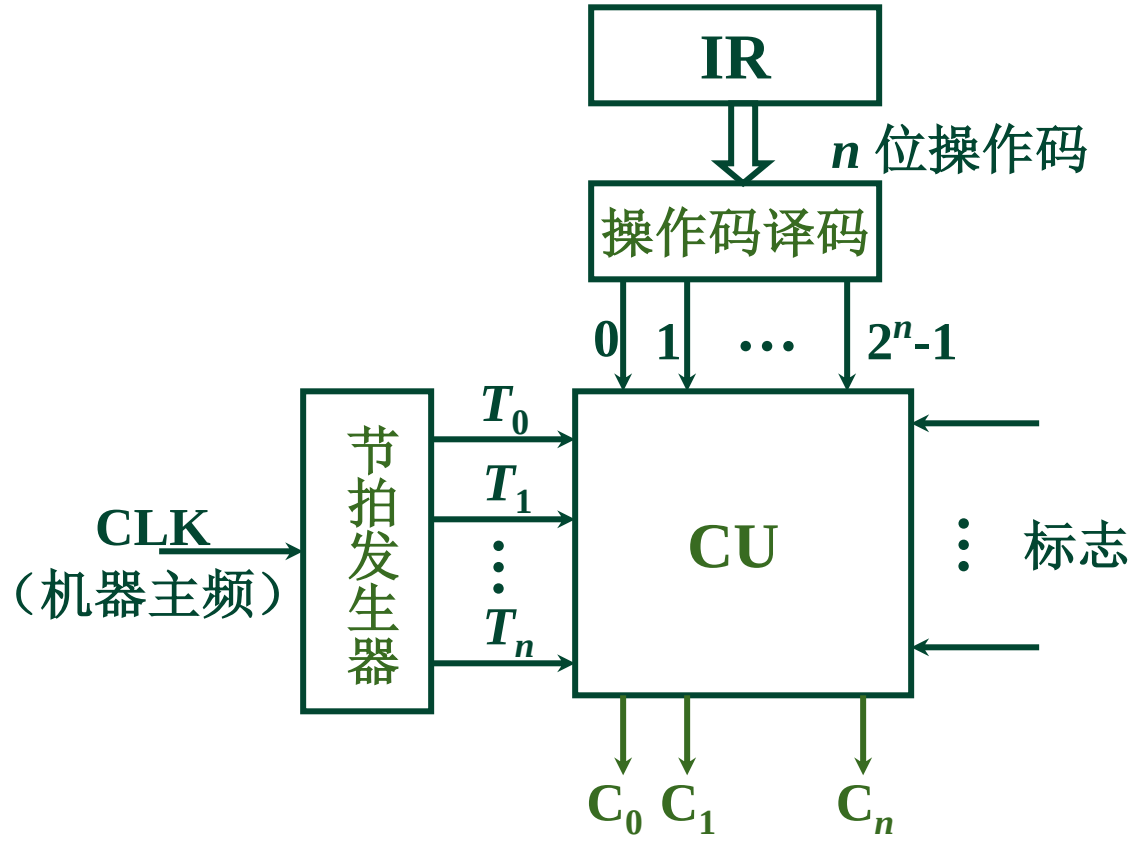
第10章　控制单元的设计

10.1　组合逻辑设计

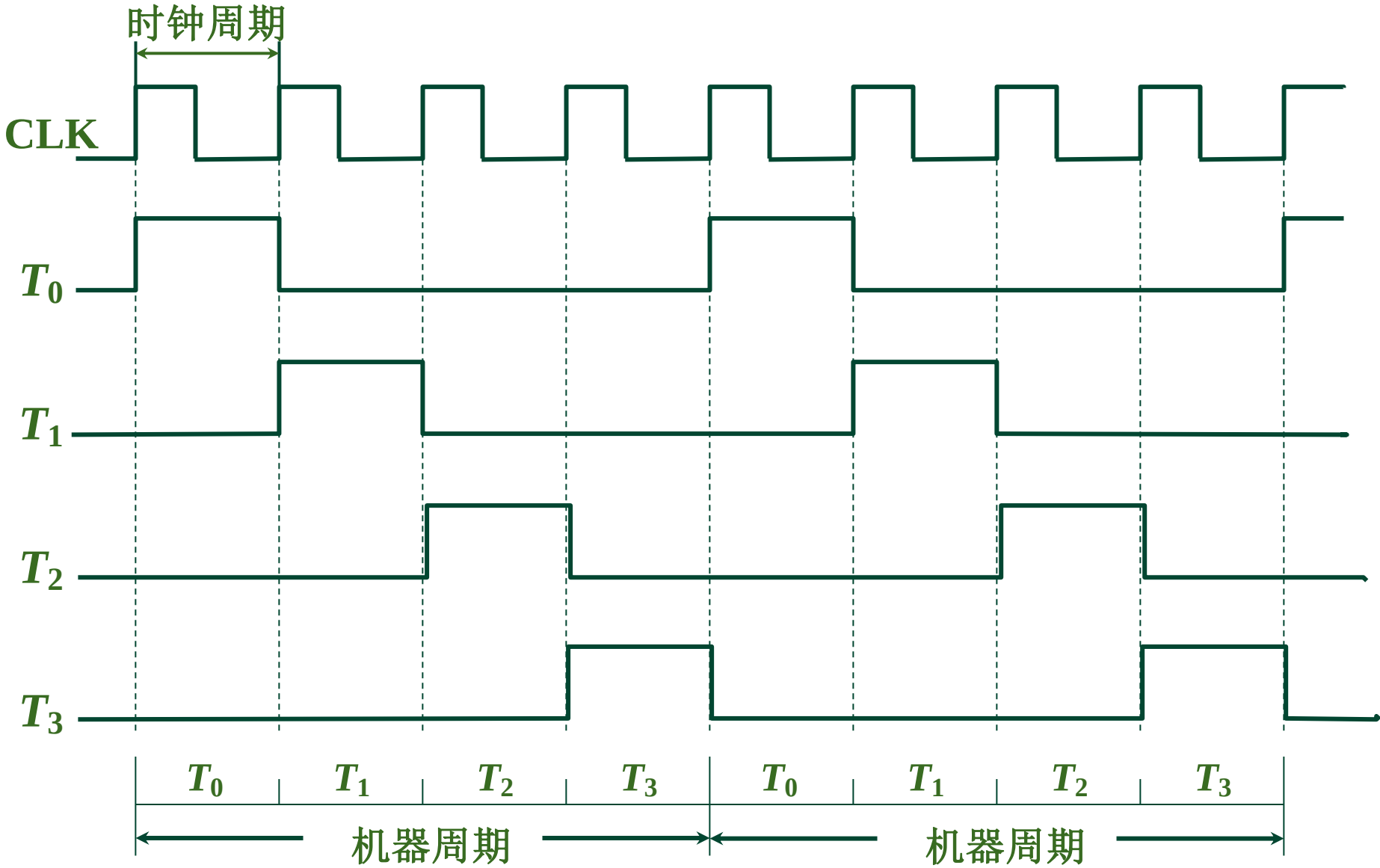
10.2　微程序设计

一、组合逻辑控制单元框图

1. CU 外特性



2. 节拍信号



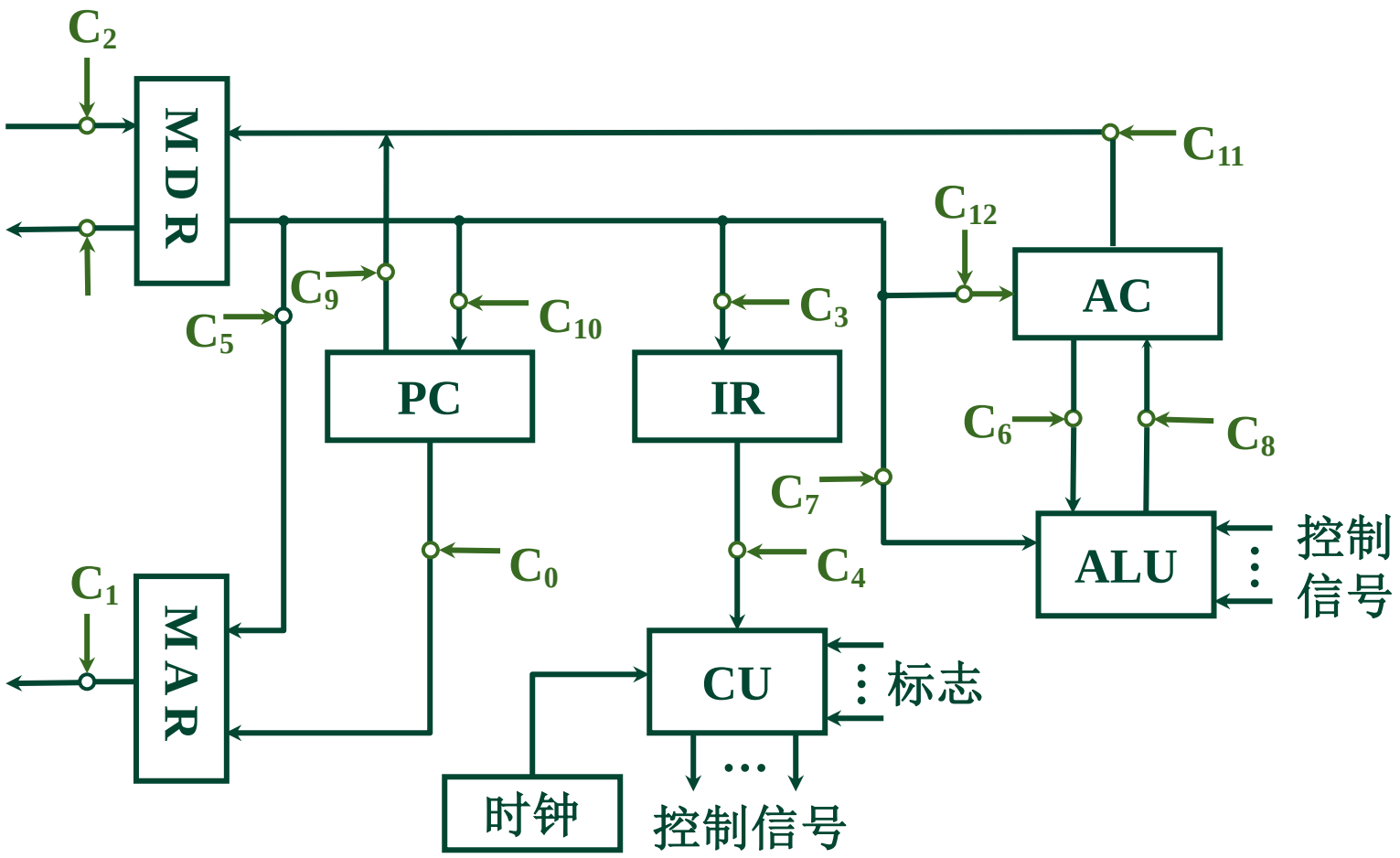
二、微操作的节拍安排

10.1

假设：采用 同步控制方式

一个 机器周期 内有 3个节拍（时钟周期）

CPU 内部结构采用非总线方式



1. 安排微操作时序的原则

原则一　微操作的 先后顺序 不得随意 更改

原则二　被控对象不同 的微操作

尽量安排在 一个节拍 内完成

原则三　占用 时间较短 的微操作

尽量 安排在 一个节拍 内完成

并允许有先后顺序

2. 取指周期 微操作的 节拍安排

T_0 PC ⟶ MAR

1 ⟶ R

T_1 M (MAR) ⟶ MDR

(PC) + 1 ⟶ PC

T_2 MDR ⟶ IR

OP (IR) ⟶ ID

原则二

原则二

原则三

3. 间址周期 微操作的 节拍安排

T_0 Ad (IR) ⟶ MAR

1 ⟶ R

T_1 M (MAR) ⟶ MDR

T_2 MDR ⟶ Ad (IR)

4. 执行周期 微操作的 节拍安排

① CLA T_0

T_1

T_2 $0 \longrightarrow AC$

② COM T_0

T_1

T_2 $\overline{AC} \longrightarrow AC$

③ SHR T_0

T_1

T_2 $L(AC) \longrightarrow R(AC)$

$AC_0 \longrightarrow AC_0$

④ CSL	T_0		
	T_1		
	T_2	R (AC) ⟶ L (AC)	$AC_0 \longrightarrow AC_n$
⑤ STP	T_0		
	T_1		
	T_2	0 ⟶ G	
⑥ ADD X	T_0	Ad (IR) ⟶ MAR	1 ⟶ R
	T_1	M (MAR) ⟶ MDR	
	T_2	(AC) + (MDR) ⟶ AC	
⑦ STA X	T_0	Ad (IR) ⟶ MAR	1 ⟶ W
	T_1	AC ⟶ MDR	
	T_2	MDR ⟶ M (MAR)	

⑧ LDA X

T_0 Ad (IR) ⟶ MAR　1 ⟶ R

T_1 M (MAR) ⟶ MDR

T_2 MDR ⟶ AC

⑨ JMP X

T_0

T_1

T_2 Ad (IR) ⟶ PC

⑩ BAN X

T_0

T_1

T_2 $A_0 \cdot Ad(IR) + \overline{A}_0 \cdot PC \longrightarrow PC$

5. 中断周期 微操作的 节拍安排

T_0　0 ⟶ MAR　　1 ⟶ W　　硬件关中断

T_1　PC ⟶ MDR

T_2　MDR ⟶ M (MAR)　　向量地址 ⟶ PC

中断隐指令完成

节拍安排例子

- 教材P398-400

例10.1~10.2

三、组合逻辑设计步骤

- 设计指令的操作码，确定指令长度是固定的还是变长的。
- 确定机器周期、节拍和时钟周期，确定机器周期是固定的还是变长的。
- 根据指令功能和CPU的结构图，绘制每条指令的微操作流程图并综合成一个总的流程图。
- 给微操作流程图安排时序，确定每条指令所需的机器周期及在各机器周期需完成的操作，排出微操作时间表。
- 根据操作时间表写出微操作的逻辑表达式，即

  微操作＝周期·节拍·时钟脉冲·指令码·其他条件
- 根据微操作的表达式，画出组合逻辑电路

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		PC⟶ MAR						
			1⟶ R						
	T_1		M(MAR) ⟶MDR						
			(PC) +1 ⟶PC						
	T_2		MDR⟶ IR						
			OP(IR) ⟶ID						
		I	1⟶ IND						
		$\bar{I}$	1⟶ EX						

间址特征

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) ⟶ MAR						
			1⟶ R						
	T_1		M(MAR) ⟶MDR						
	T_2		MDR⟶ Ad (IR)						
		$\overline{\text{IND}}$	1⟶ EX						

间址周期标志

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) ⟶ MAR						
			1 ⟶ R						
			1 ⟶ W						
	T_1		M(MAR) ⟶ MDR						
			AC ⟶ MDR						
	T_2		(AC)+(MDR) →AC						
			MDR ⟶ M(MAR)						
			MDR ⟶ AC						
			0 ⟶ AC						

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		PC⟶ MAR	1	1	1	1	1	1
			1⟶ R	1	1	1	1	1	1
	T_1		M(MAR) ⟶MDR	1	1	1	1	1	1
			(PC) +1⟶ PC	1	1	1	1	1	1
	T_2		MDR⟶ IR	1	1	1	1	1	1
			OP(IR)⟶ ID	1	1	1	1	1	1
		I	1⟶ IND			1	1	1	1
		$\bar{I}$	1⟶ EX	1	1	1	1	1	1

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) ⟶ MAR			1	1	1	1
			1⟶ R			1	1	1	1
	T_1		M(MAR) ⟶MDR			1	1	1	1
	T_2		MDR⟶ Ad (IR)			1	1	1	1
		$\overline{\mathbf{IND}}$	1⟶ EX			1	1	1	1

三、组合逻辑设计步骤

1. 列出操作时间表

教材P402

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) ⟶ MAR			1	1	1	
			1⟶ R			1		1	
			1 ⟶ W				1		
	T_1		M(MAR) ⟶MDR			1		1	
			AC⟶ MDR				1		
	T_2		(AC)+(MDR)→AC			1			
			MDR⟶ M(MAR)				1		
			MDR ⟶AC					1	
			0⟶ AC	1					

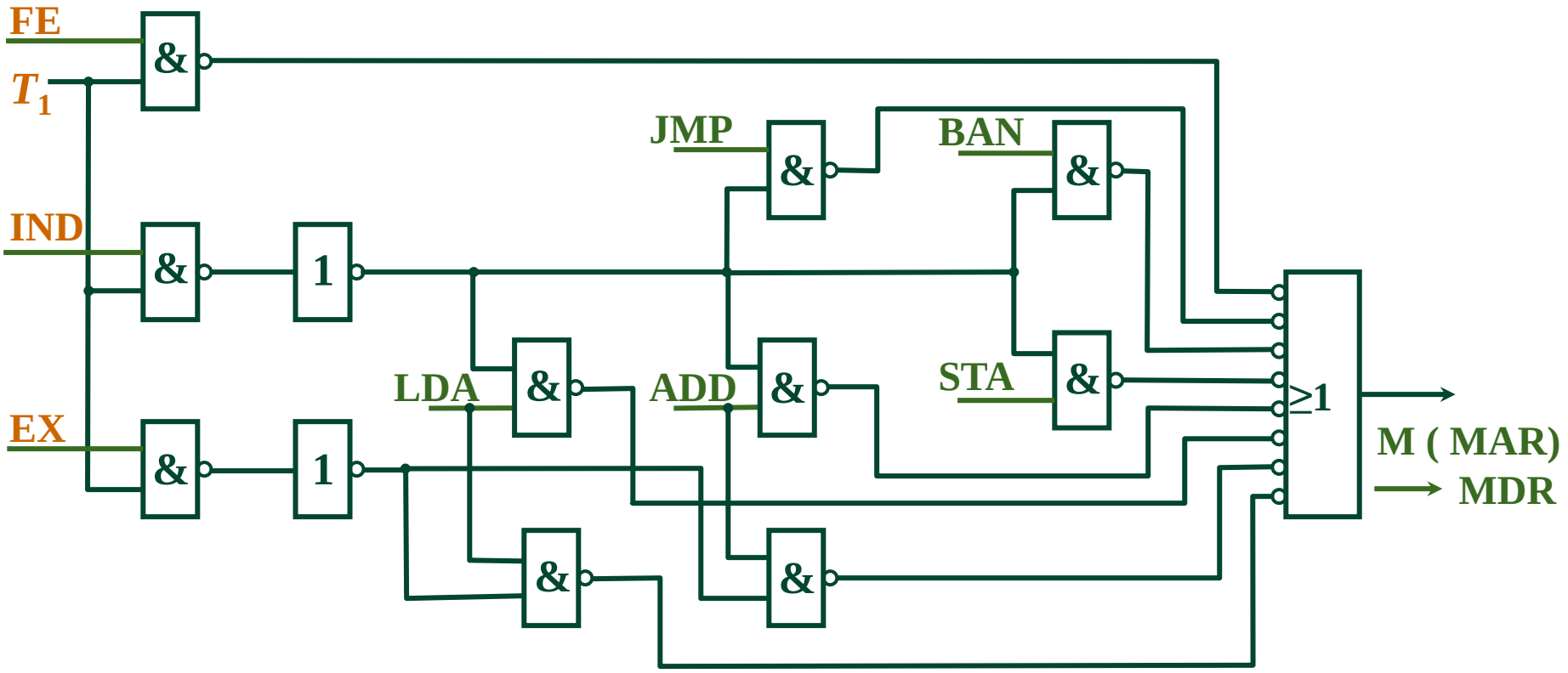
2. 写出微操作命令的最简表达式

$$M(MAR) \longrightarrow MDR$$

$$= FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) + EX \cdot T_1 (ADD + LDA)$$

$$= T_1\{FE + IND (ADD + STA + LDA + JMP + BAN) + EX (ADD + LDA)\}$$

3. 画出逻辑图



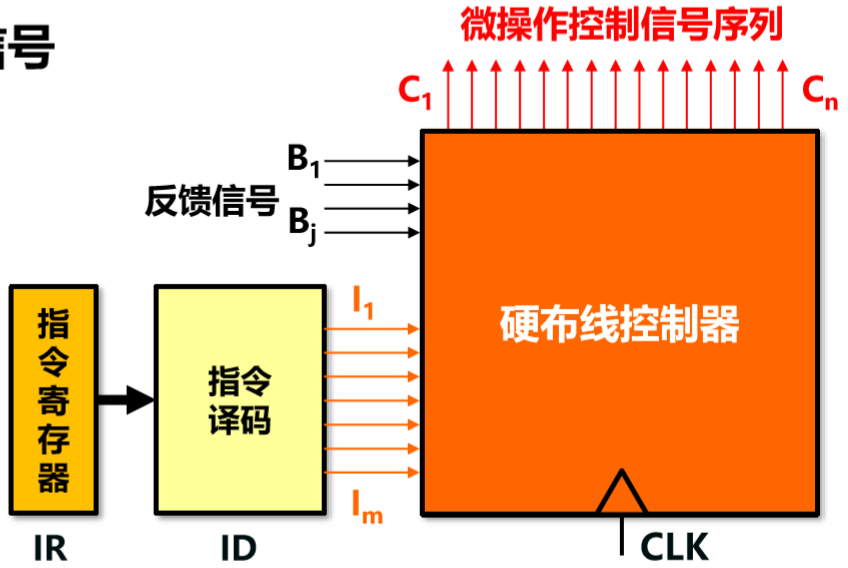
特点

- 思路清晰，简单明了
- 庞杂，调试困难，修改困难
- 速度快 （RISC）

实例：单总线CPU硬布线设计流程*

1 基本原理

- 将控制器看成产生固定时序控制信号的逻辑电路
- 输入信号：指令译码，时钟信号，反馈信号
- 输出信号：功能部件控制信号序列
- 设计目标：最少元件，最快速度
- 理论基础：布尔代数
- 组成器件：门电路，触发器

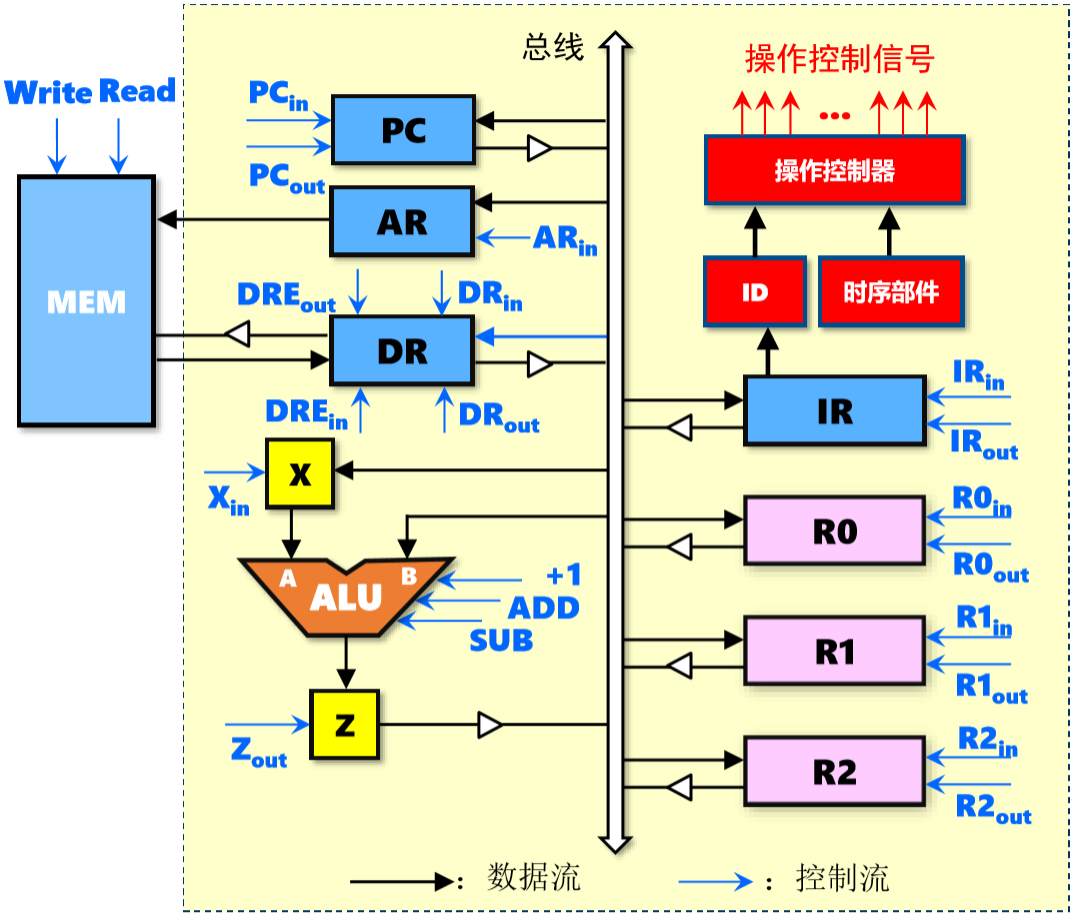


机器指令字 → 控制器信号序列

实例：单总线CPU硬布线设计流程

2 单总线结构CPU

1. LOAD R0,6#
2. MOVE R1,10
3. ADD R0,R1
4. STORE R0,(R2)
5. JMP 1000



实例：单总线CPU硬布线设计流程

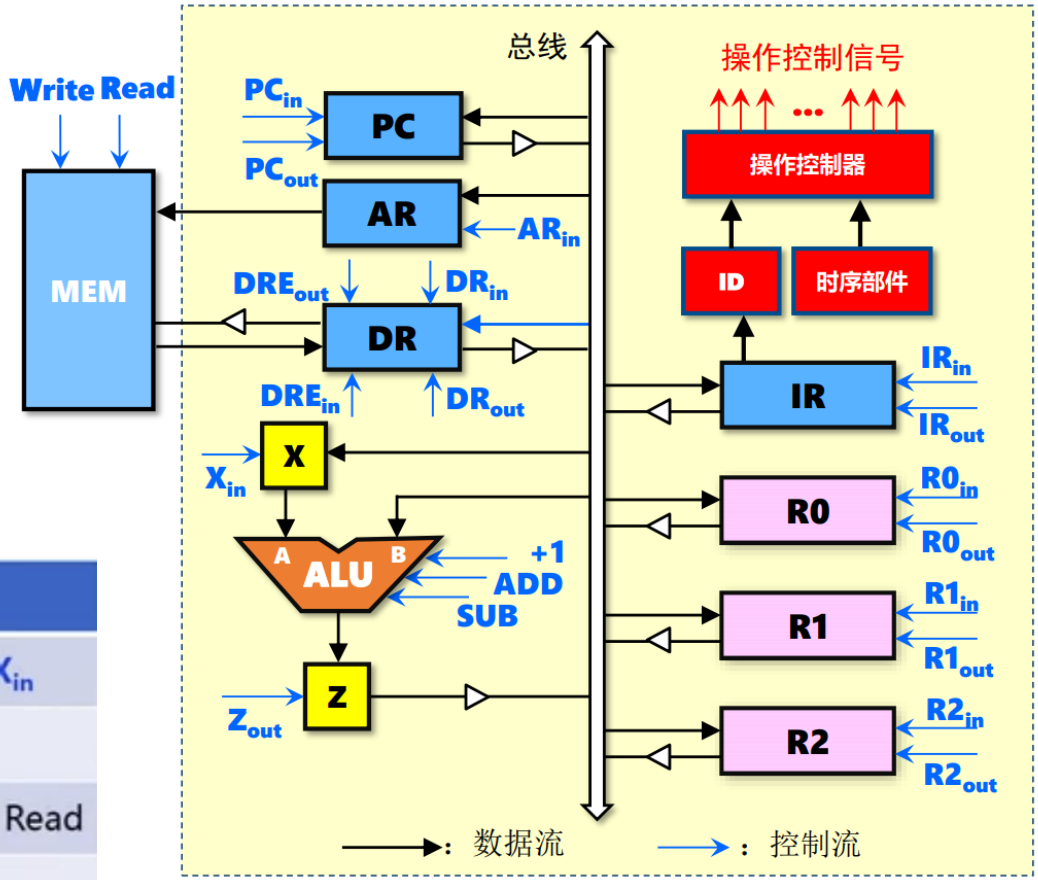
#	指令	指令功能
1	LOAD R0,6#	Mem[6] → R0
2	MOVE R1,10	10 → R1
3	ADD R0,R1	(R0) + (R1) → R0
4	STORE R0,(R2)	(R0) → Mem[(R2)]
5	JMP 1000	1000 → PC

实例：单总线CPU硬布线设计流程

取指令数据通路

Mem[PC++] → IR

节拍	数据通路	控制信号
T1	(PC)→AR, (PC)→X	PC_{out}, AR_{in}, X_{in}
T2	(X)+1→Z	+1, Read
T3	(Z)→PC, Mem[AR]→DR	Z_{out}, PC_{in}, DRE_{in}, Read
T4	(DR)→IR	DR_{out}, IR_{in}



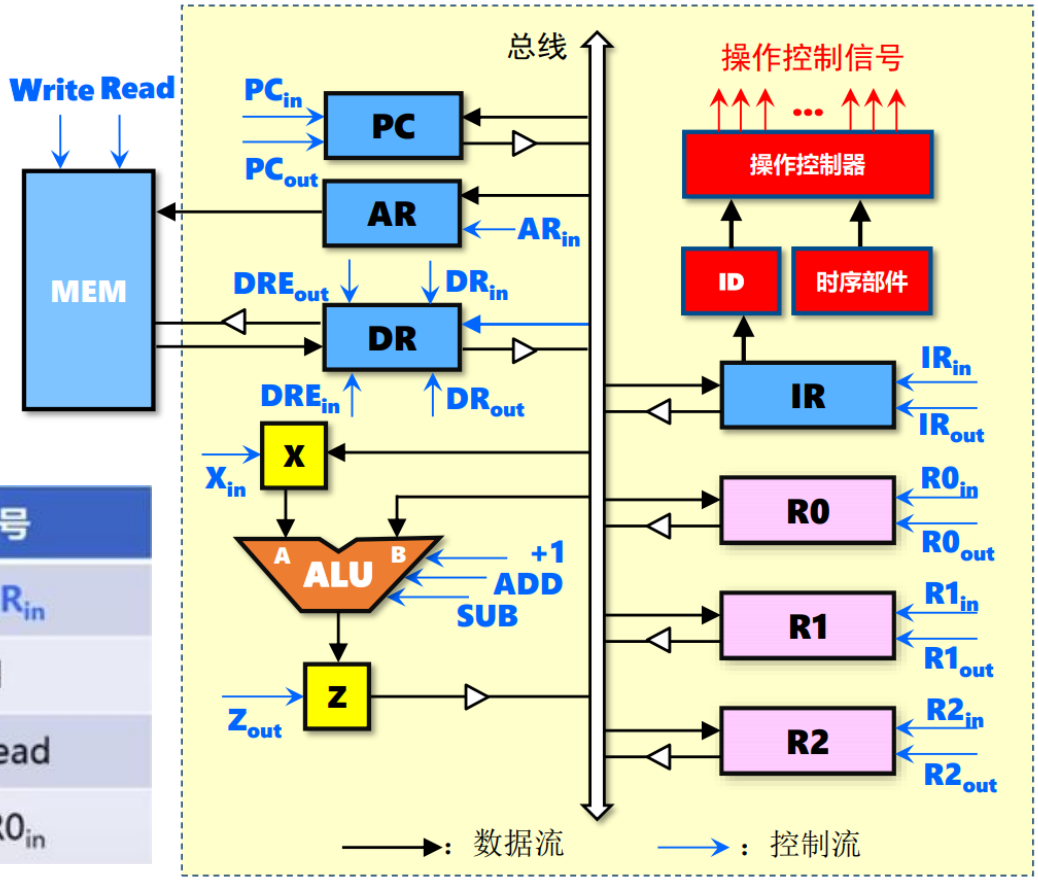
实例：单总线CPU硬布线设计流程

LOAD指令执行数据通路

LOAD R0,6#

Mem[IR_A] → Reg

节拍	数据通路	控制信号
T1	(IR_A)→AR	IR_{out}, AR_{in}
T2		Read
T3	Mem[AR]→DR	DRE_{in},Read
T4	(DR)→R0	DR_{out}, $R0_{in}$

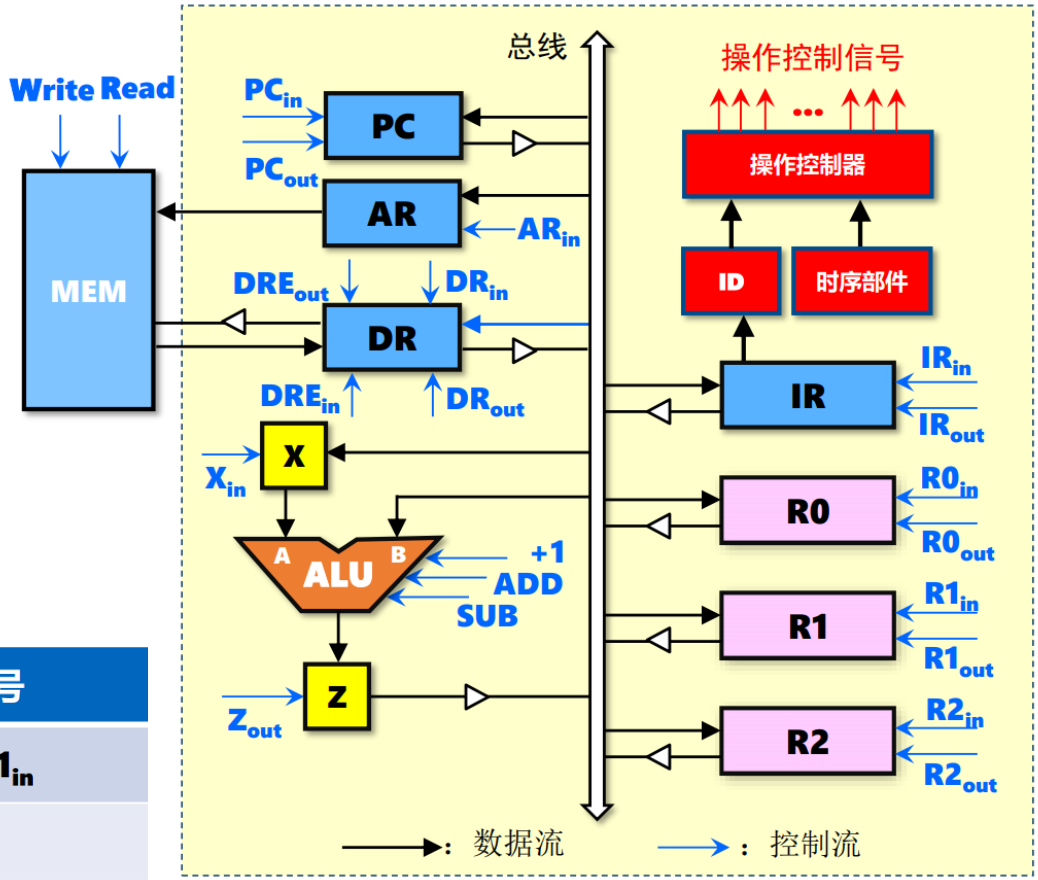


实例：单总线CPU硬布线设计流程

MOVE指令执行数据通路

MOVE R1,10

$(IR_A) \rightarrow Reg$



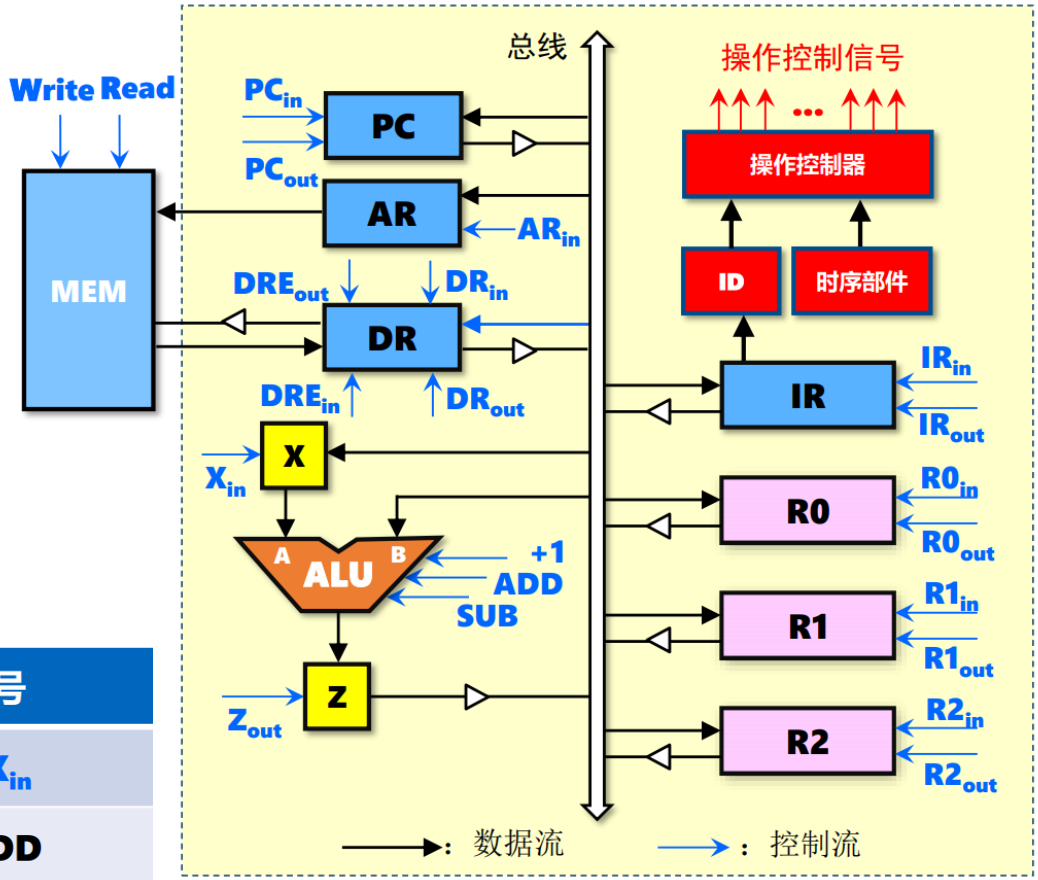
节拍	数据通路	控制信号
T1	$(IR_A) \rightarrow R[1]$	IR_{out}, $R1_{in}$
T2		
T3		
T4		

实例：单总线CPU硬布线设计流程

ADD指令执行数据通路

ADD R0,R1

(R0)+(R1) → R0



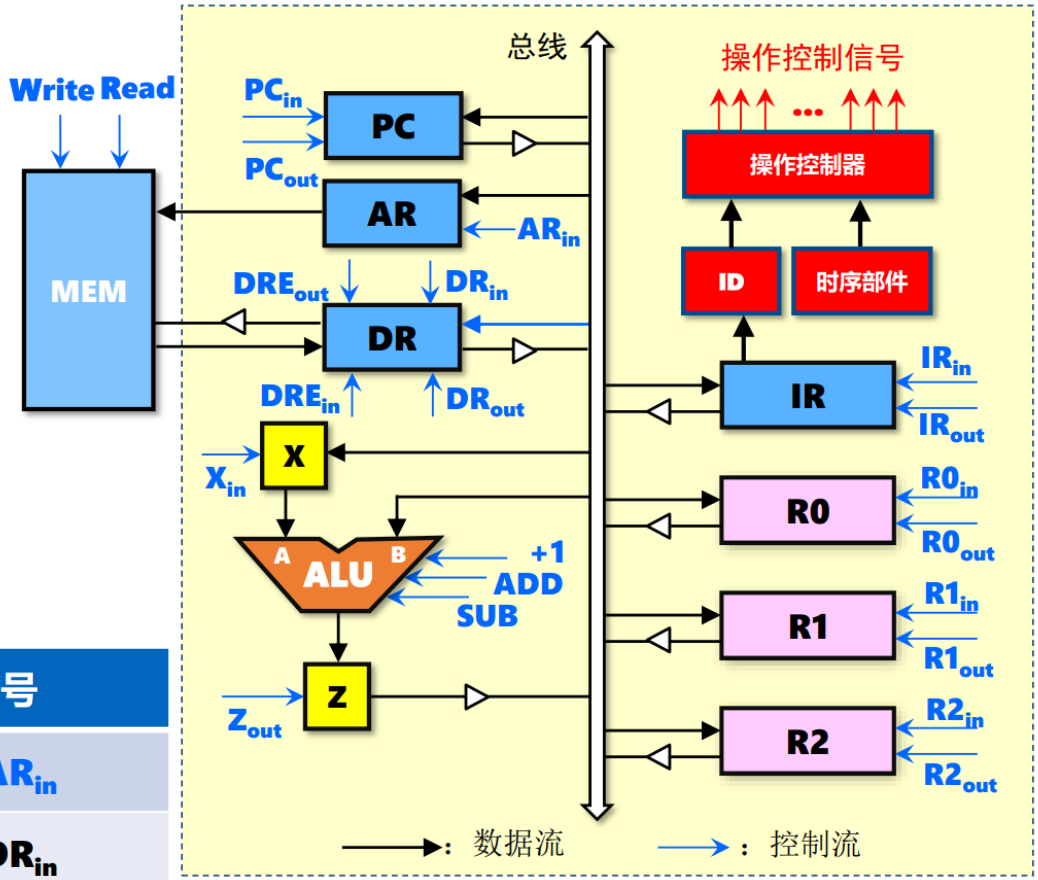
节拍	数据通路	控制信号
T1	(R0)→X	$R0_{out}$, X_{in}
T2	(X)+(R1)→Z	$R1_{out}$,ADD
T3	(Z)→R0	Z_{out},$R0_{in}$
T4		

实例：单总线CPU硬布线设计流程

STORE指令 数据通路

STORE R0,(R2)

(R0) → Mem[R2]



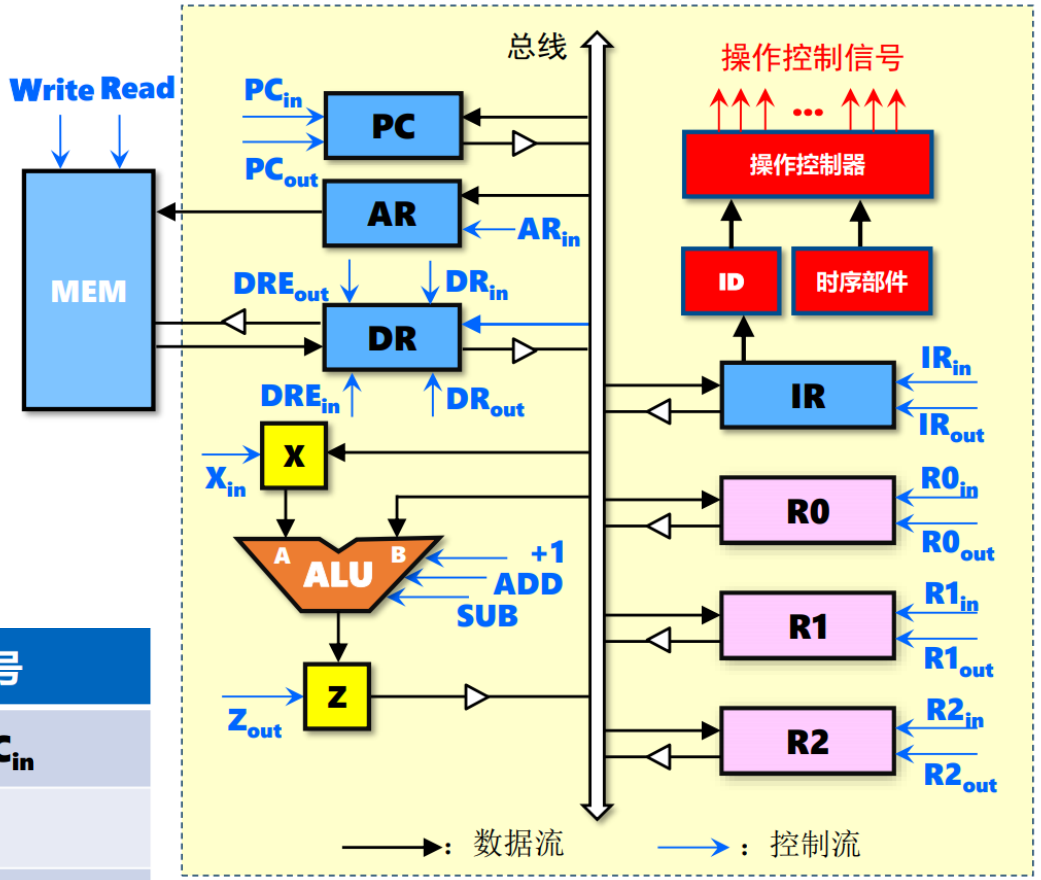
节拍	数据通路	控制信号
T1	(R2)→AR	$R2_{out}$, AR_{in}
T2	(R0)→DR	$R0_{out}$, DR_{in}
T3	(DR) →Mem[AR]	DRE_{out}, Write
T4		

实例：单总线CPU硬布线设计流程

JMP指令数据通路

JMP 1000

(IR_A) → PC



节拍	数据通路	控制信号
T1	(IR_A)→PC	IR_{out}, PC_{in}
T2		
T3		
T4		

实例：单总线CPU硬布线设计流程

3 单总线结构CPU指令周期

取指令周期

节拍	控制信号(4 cycles)
T1	PC_{out}, AR_{in}, X_{in}
T2	+1, Read
T3	Z_{out}, PC_{in}, DRE_{in}, Read
T4	DR_{out}, IR_{in}

- 定长指令周期：传统三级时序
  - 2个机器周期，8个时钟周期、慢、设计简单
- 变长指令周期：现代时序
  - 时钟周期数可变，快，设计复杂

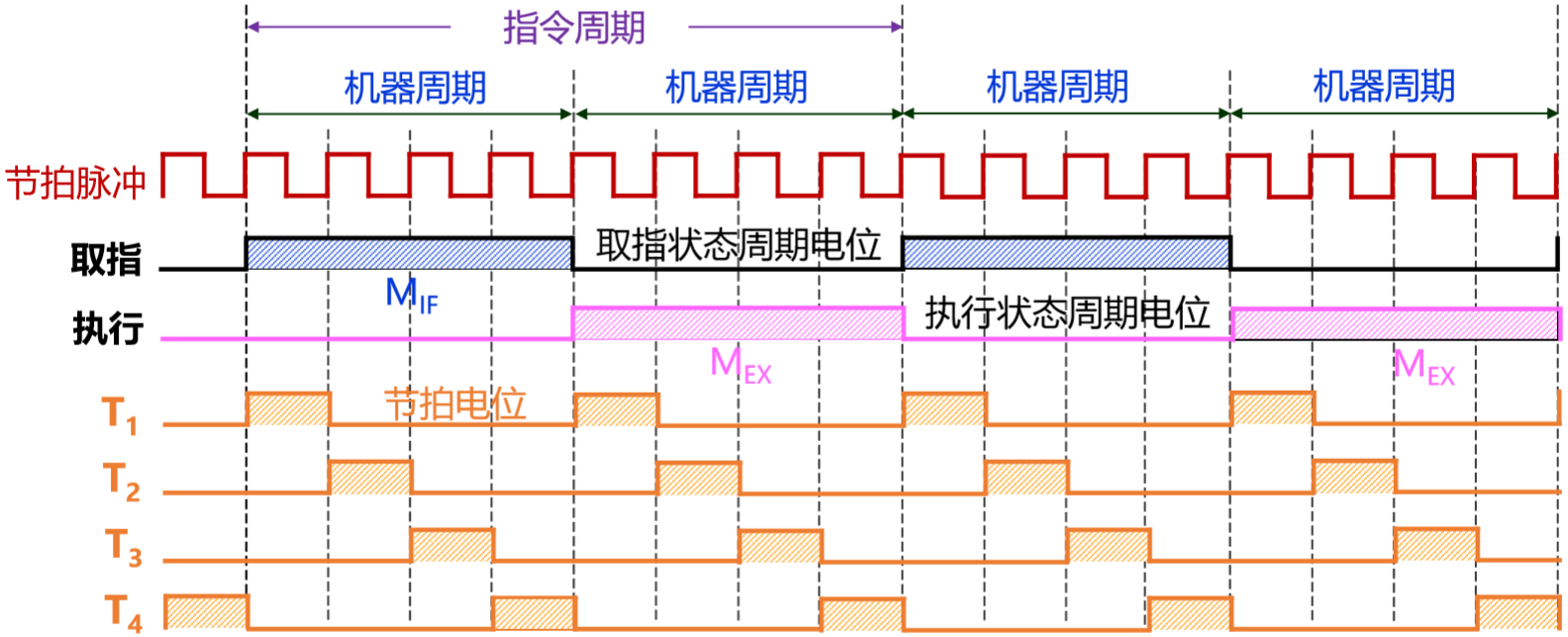
执行周期

节拍	LOAD (4 cycles)	MOVE (1 cycles)	ADD (3 cycles)	STORE (3 cycles)	JMP (1 cycles)
T5	IR_{out}, AR_{in}	IR_{out}, $R1_{in}$	$R0_{out}$, X_{in}	$R2_{out}$, AR_{in}	IR_{out}, PC_{in}
T6	Read		$R1_{out}$,ADD	$R0_{out}$, DR_{in}	
T7	DRE_{in},Read		Z_{out},$R0_{in}$	DRE_{out}, Write	
T8	DR_{out}, $R0_{in}$				

实例：单总线CPU硬布线设计流程

4 定长指令周期时序产生器　传统三级时序

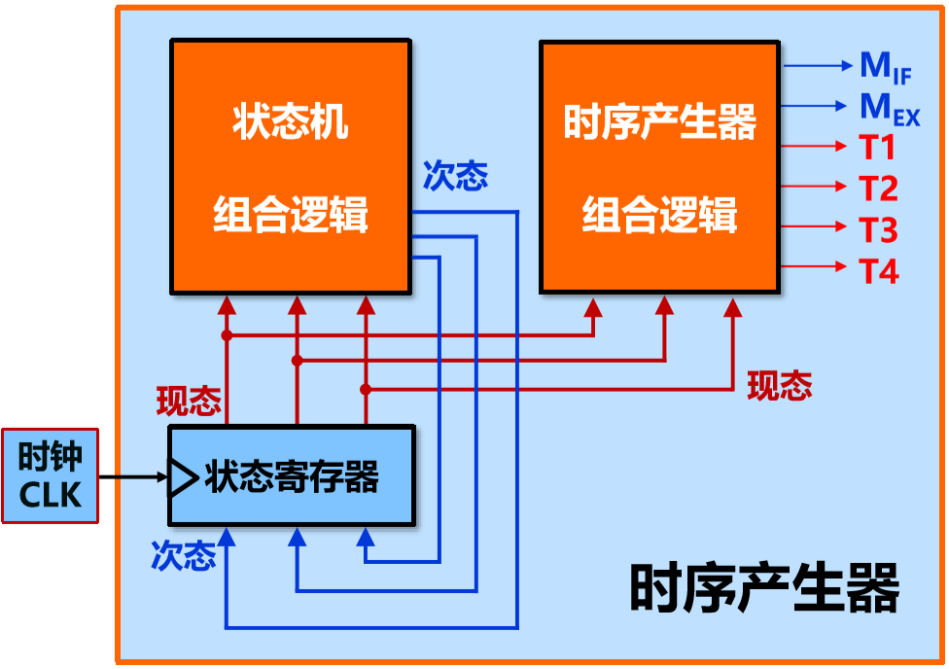
固定2个机器周期，8个时钟节拍

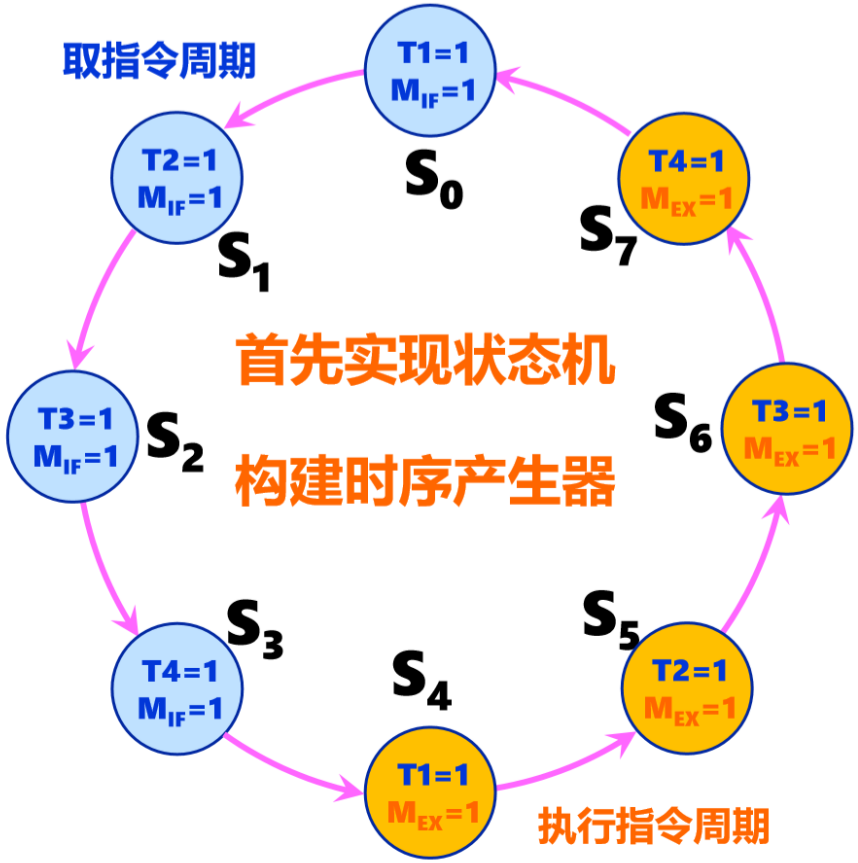


构建时序产生器　输出：M_{IF}，M_{EX}，T1，T2，T3，T4

实例：单总线CPU硬布线设计流程

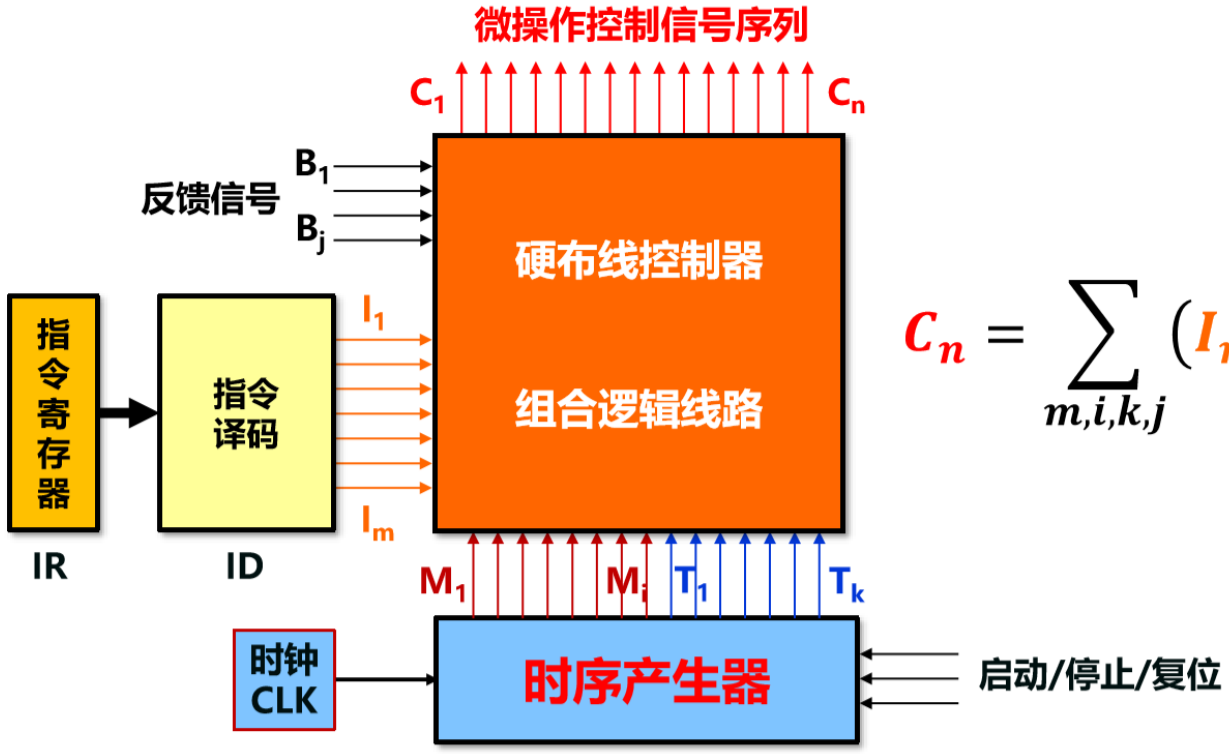
5 时序产生器状态机





实例：单总线CPU硬布线设计流程

6 硬布线控制器基本架构



$$C_n = \sum_{m,i,k,j} \left(I_m \cdot M_i \cdot T_k \cdot B_j \right)$$

时序产生器循环产生周期电位、节拍电位，供控制器对信号进行时间调制

实例：单总线CPU硬布线设计流程

7 单总线CPU控制信号生成

取指令周期 M_{IF}

节拍	控制信号
T1	PC_{out}, AR_{in}, X_{in}
T2	+1, Read
T3	Z_{out}, PC_{in}, DRE_{in}, Read
T4	DR_{out}, IR_{in}

$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

- $Read = M_{IF} \cdot (T2+T3) + LOAD \cdot M_{EX} \cdot (T2+T3)$
- $AR_{in} = M_{IF} \cdot T1 + (LOAD+STORE) \cdot M_{EX} \cdot T1$

执行周期 M_{EX}

节拍	LOAD	MOVE	ADD	STORE	JMP
T1	IR_{out}, AR_{in}	IR_{out}, $R1_{in}$	$R0_{out}$, X_{in}	$R2_{out}$, AR_{in}	IR_{out}, PC_{in}
T2	Read		$R1_{out}$, ADD	$R0_{out}$, DR_{in}	
T3	DRE_{in}, Read		Z_{out}, $R0_{in}$	DRE_{out}, Write	
T4	DR_{out}, $R0_{in}$				

实例：单总线CPU硬布线设计流程

8 固定指令周期硬布线控制器设计过程

1. 设计三级时序产生器： 所有指令固定机器周期数，节拍数，
2. 列出所有机器指令的指令周期流程图，明确每个节拍的控制信号；
3. 找出产生同一微操作控制信号的条件；
4. 写出各微操作控制信号的布尔表达式；
5. 化简各表达式；
6. 利用组合逻辑电路实现。

$$C_n = \sum_i \left(M_i \cdot T_k \cdot B_j \cdot \sum_m I_m \right)$$

单选题 1分

设置

【2016统考真题】单周期处理器中所有指令的指令周期为一个时钟周期。下列关于单周期处理器的叙述中，错误的是(　)。

A 可以采用单总线结构数据通路

B 处理器时钟频率较低

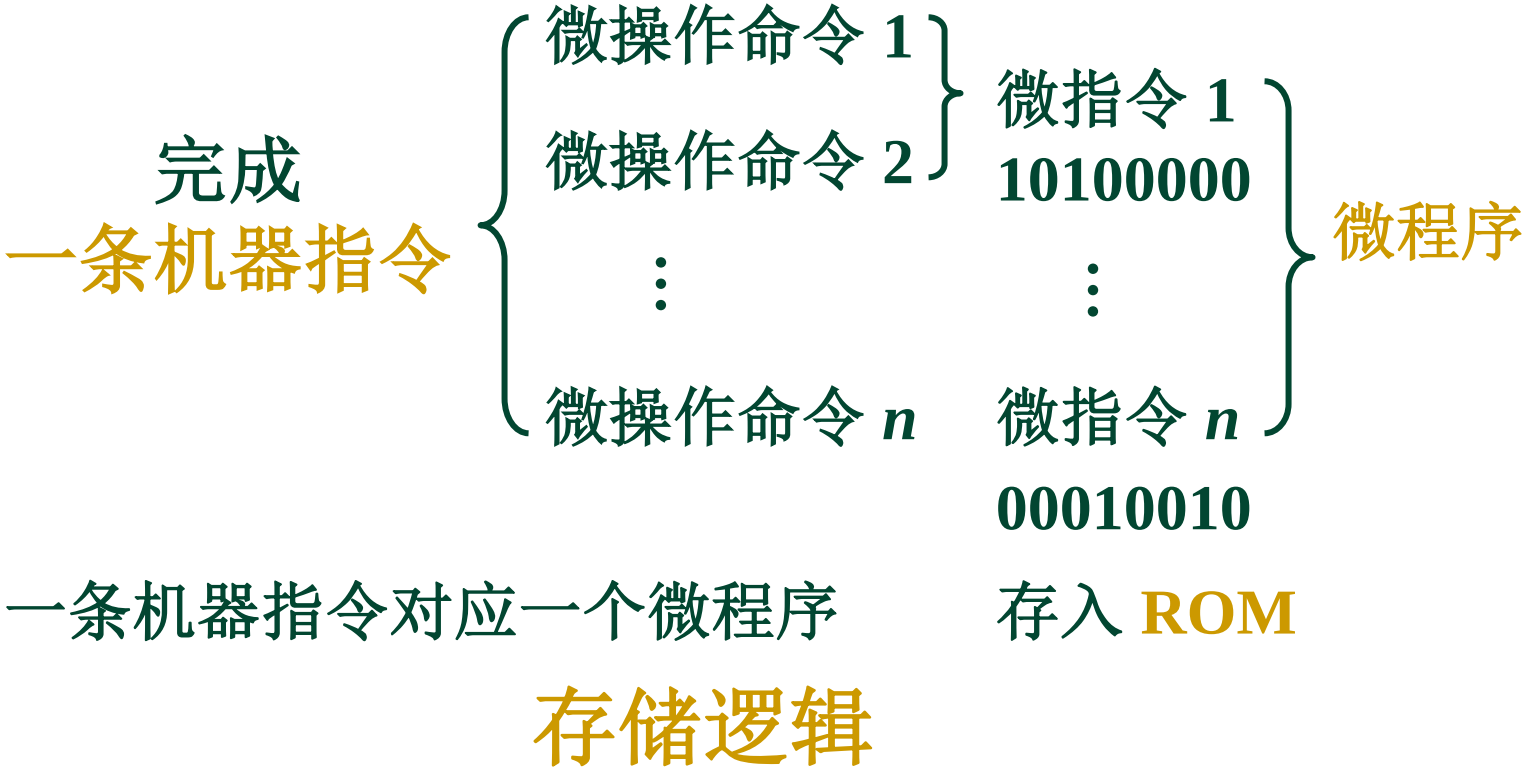
C 在指令执行过程中控制信号不变

D 每条指令的CPI为1

提交

一、微程序设计思想的产生

1951 英国剑桥大学教授 Wilkes



存储逻辑

微指令的格式

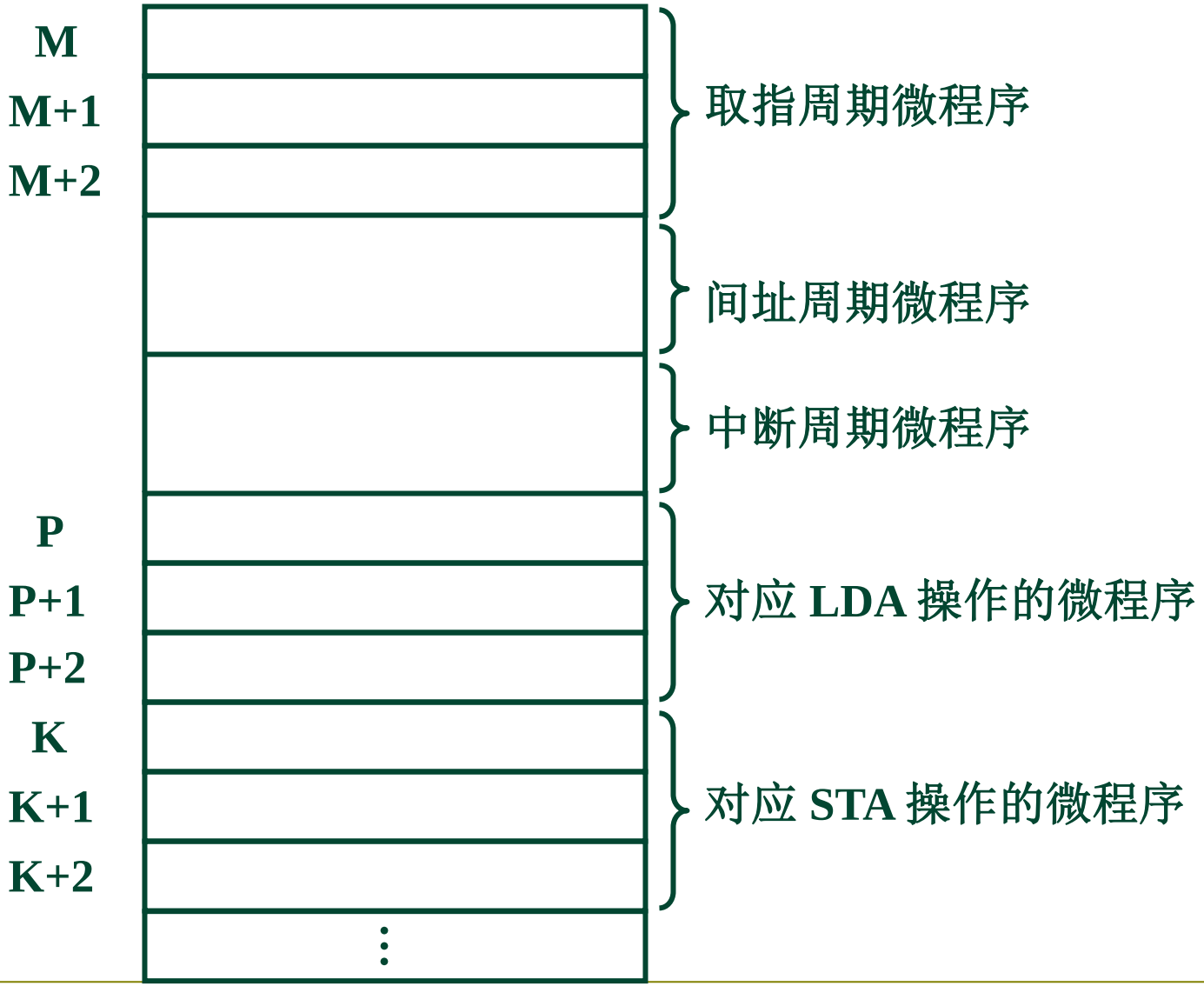
微指令：

控制字段	下址字段

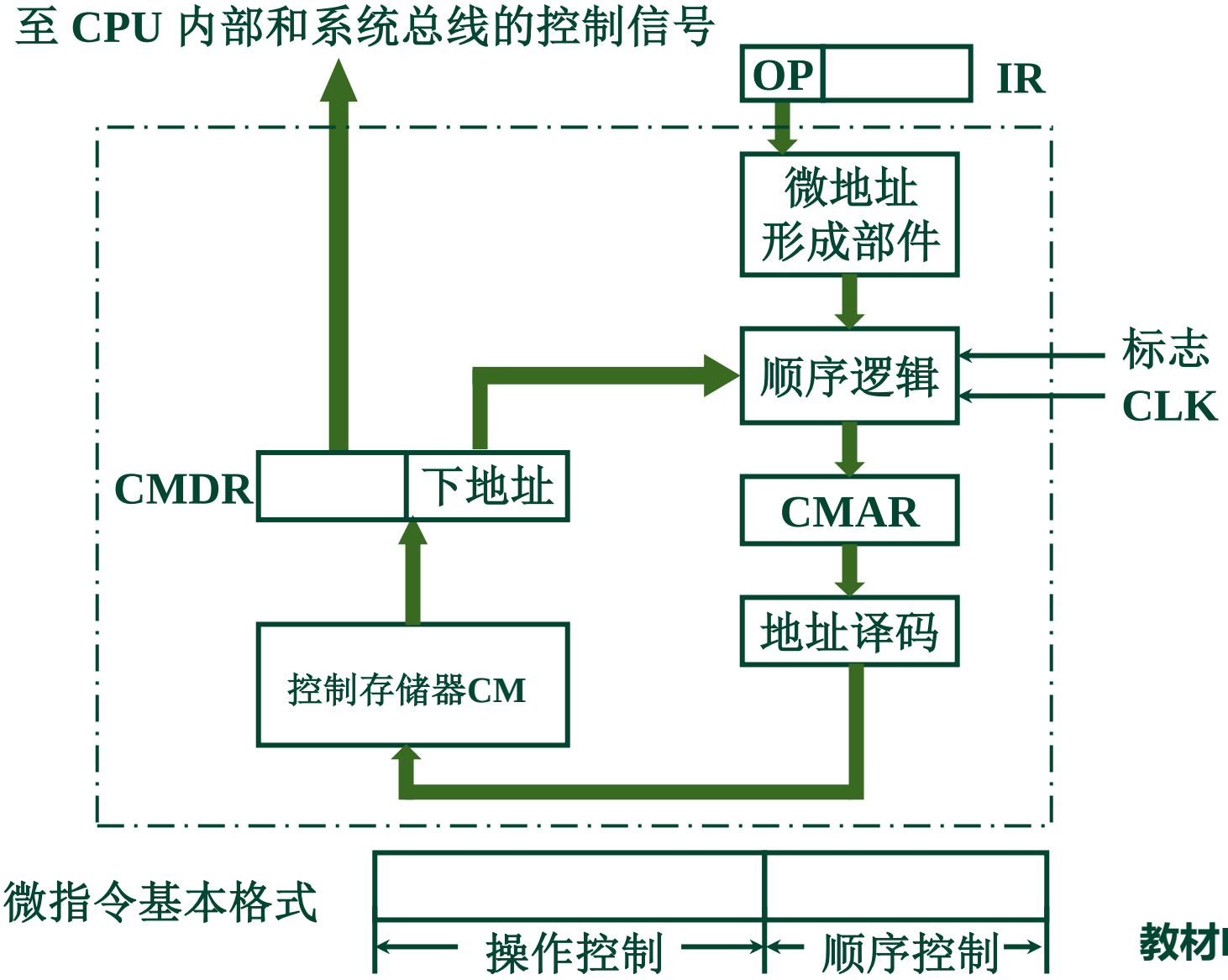
- 控制字段：操作控制，发出各种控制信号
- 下址字段：顺序控制，指出下条微指令地址，以控制微指令序列的执行顺序

二、微程序控制单元框图及工作原理

1. 机器指令对应的微程序

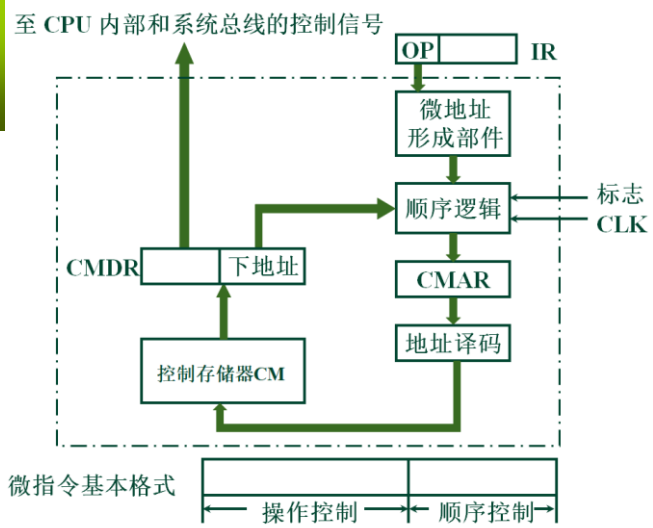


2. 微程序控制单元的基本框图



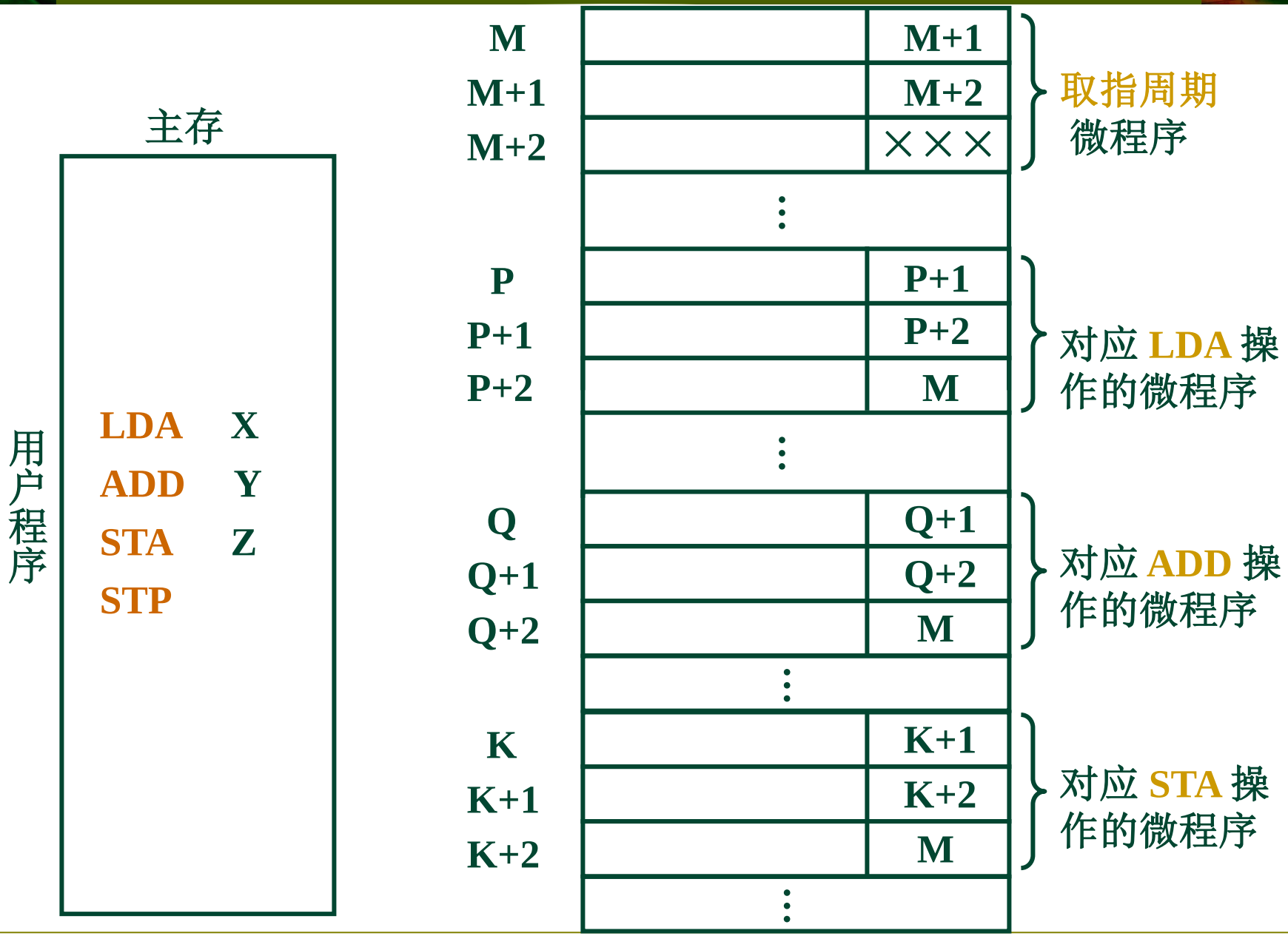
教材P405

二、微程序控制单元框图及工作原理

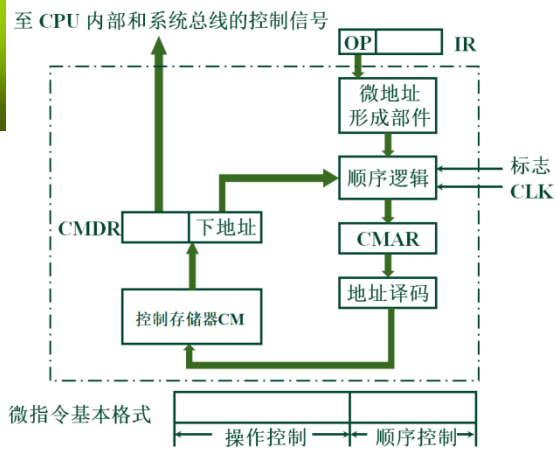


M	M+1	取指周期 微程序
M+1	M+2	
M+2	×××	
	转执行周期微程序	间址周期 微程序
	转取指周期微程序	中断周期 微程序
P	P+1	对应 LDA 操作的微程序
P+1	P+2	
P+2	M	
K	K+1	对应 STA 操作的微程序
K+1	K+2	
K+2	M	
	⋮	

3. 工作原理


控存
主存
用户程序
LDA X
ADD Y
STA Z
STP
M
M+1
M+2
M+1
M+2
×××
取指周期微程序
P
P+1
P+2
P+1
P+2
M
对应 LDA 操作的微程序
Q
Q+1
Q+2
Q+1
Q+2
M
对应 ADD 操作的微程序
K
K+1
K+2
K+1
K+2
M
对应 STA 操作的微程序

3. 工作原理



(1) 取指阶段　　执行 取指周期微程序

M ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令（控制信号）

形成下条微指令地址　M + 1



Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令（控制信号）

形成下条微指令地址　M + 2



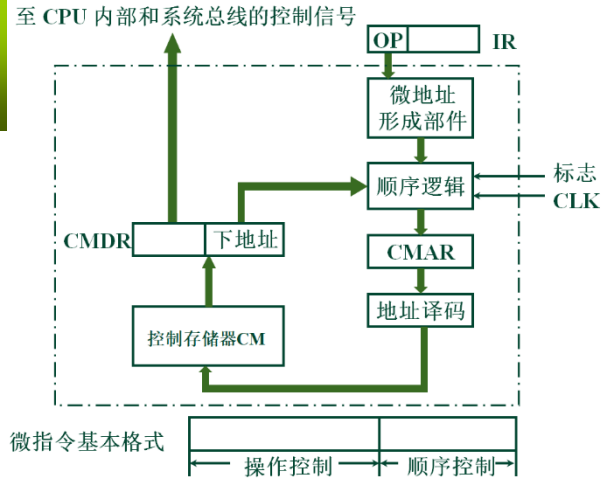
Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令（控制信号）



(2) 执行阶段　执行 LDA 微程序



OP (IR) ⟶ 微地址形成部件 ⟶ CMAR

(P ⟶ CMAR)

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址 P + 1

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址 P + 2

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址 M

Ad (CMDR) ⟶ CMAR

Ad (IR) ⟶ MAR　　1 ⟶ R



M (MAR) ⟶ MDR



MDR ⟶ AC



(M ⟶ CMAR)

(3) 取指阶段　执行取指微程序

M ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

⋮



全部微指令存在 CM 中，程序执行过程中 只需读出

关键

- 微指令的 操作控制字段如何形成微操作命令
- 微指令的 后续地址如何形成

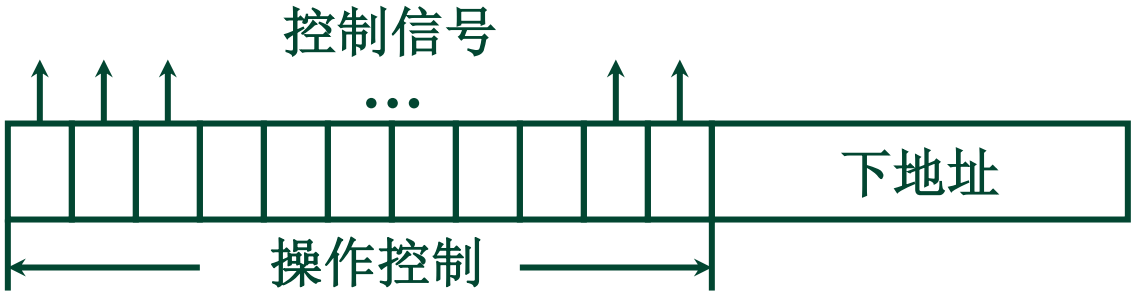
1. 直接编码（直接控制）方式



在微指令的操作控制字段中，

每一位代表一个微操作命令





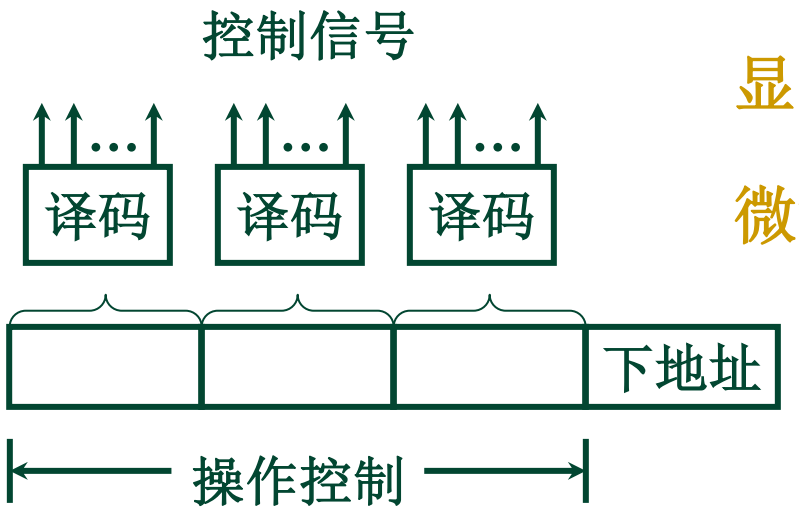
速度最快

某位为“1”表示该控制信号有效

2.字段直接编码方式

将微指令的控制字段分成若干 “段”，

每段经译码后发出控制信号



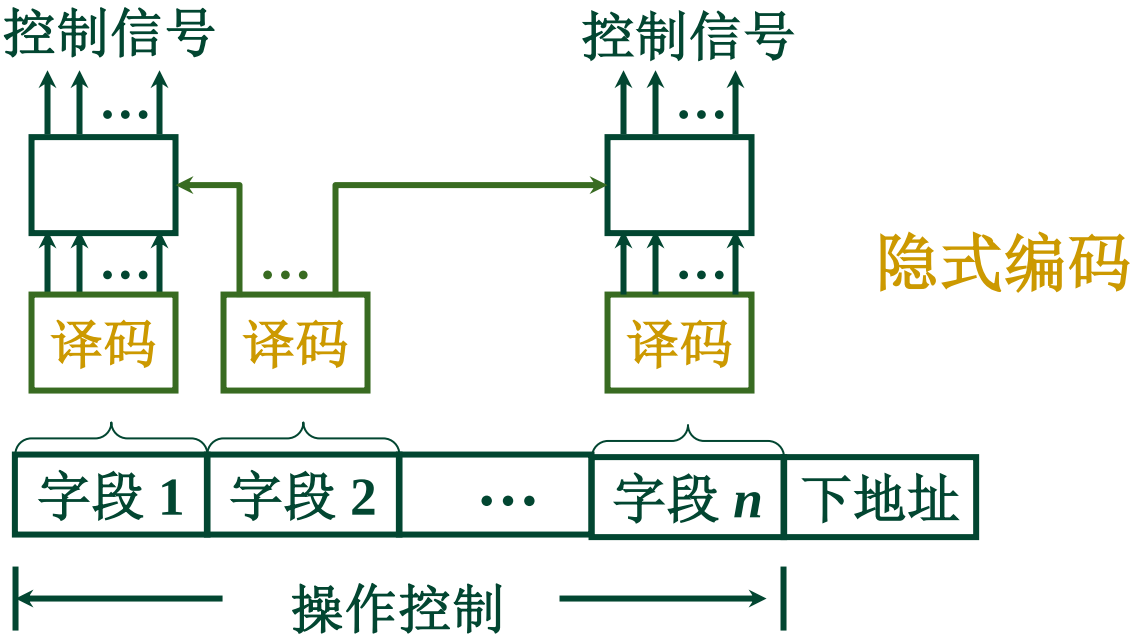
显式编码

微程序执行速度较慢

每个字段中的命令是 互斥 的

缩短 了微指令 字长，增加 了译码 时间

3. 字段间接编码方式



4. 混合编码

直接编码和字段编码（直接和间接）混合使用

5. 其他

四、微指令序列地址的形成

1. 微指令的 下地址字段 指出 （断定方式）

2. 根据机器指令的 操作码 形成：根据机器指令的操作码，由微地址形成部件形成对应该机器指令微程序的首地址。

3. 增量计数器（顺序地址）

$$(CMAR) + 1 \longrightarrow CMAR$$

4. 分支转移（转移指令）

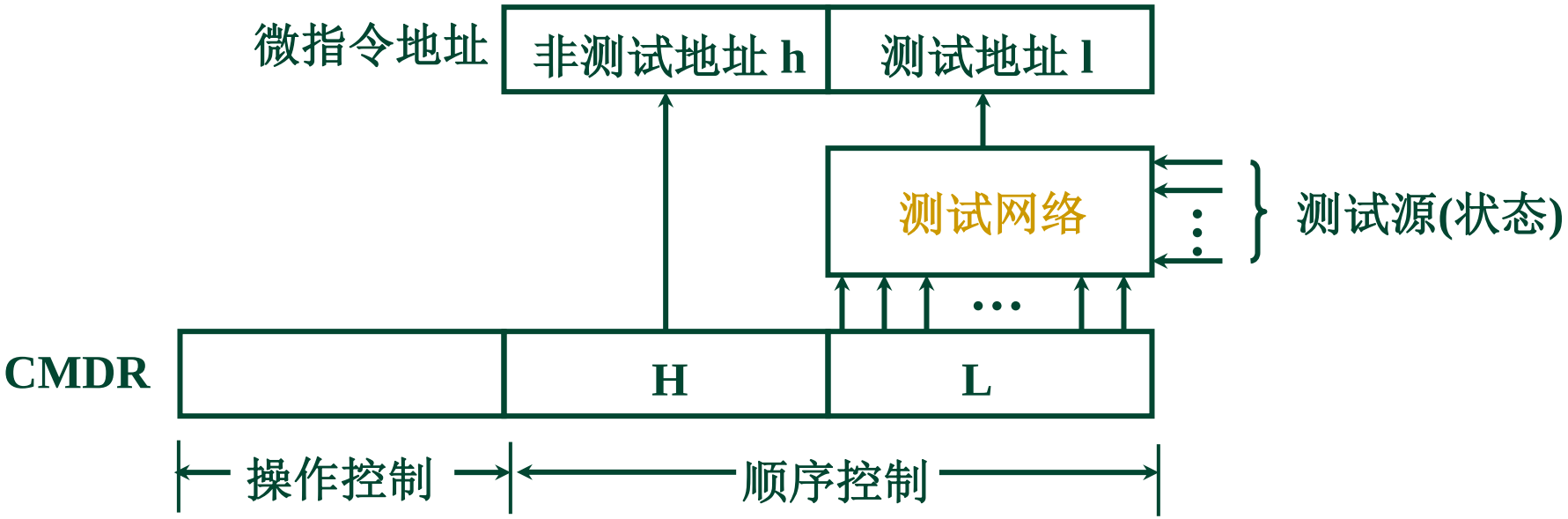
操作控制字段	转移方式	转移地址

转移方式　　指明判别条件

转移地址　　指明转移成功后的去向

四、微指令序列地址的形成

5. 通过测试网络



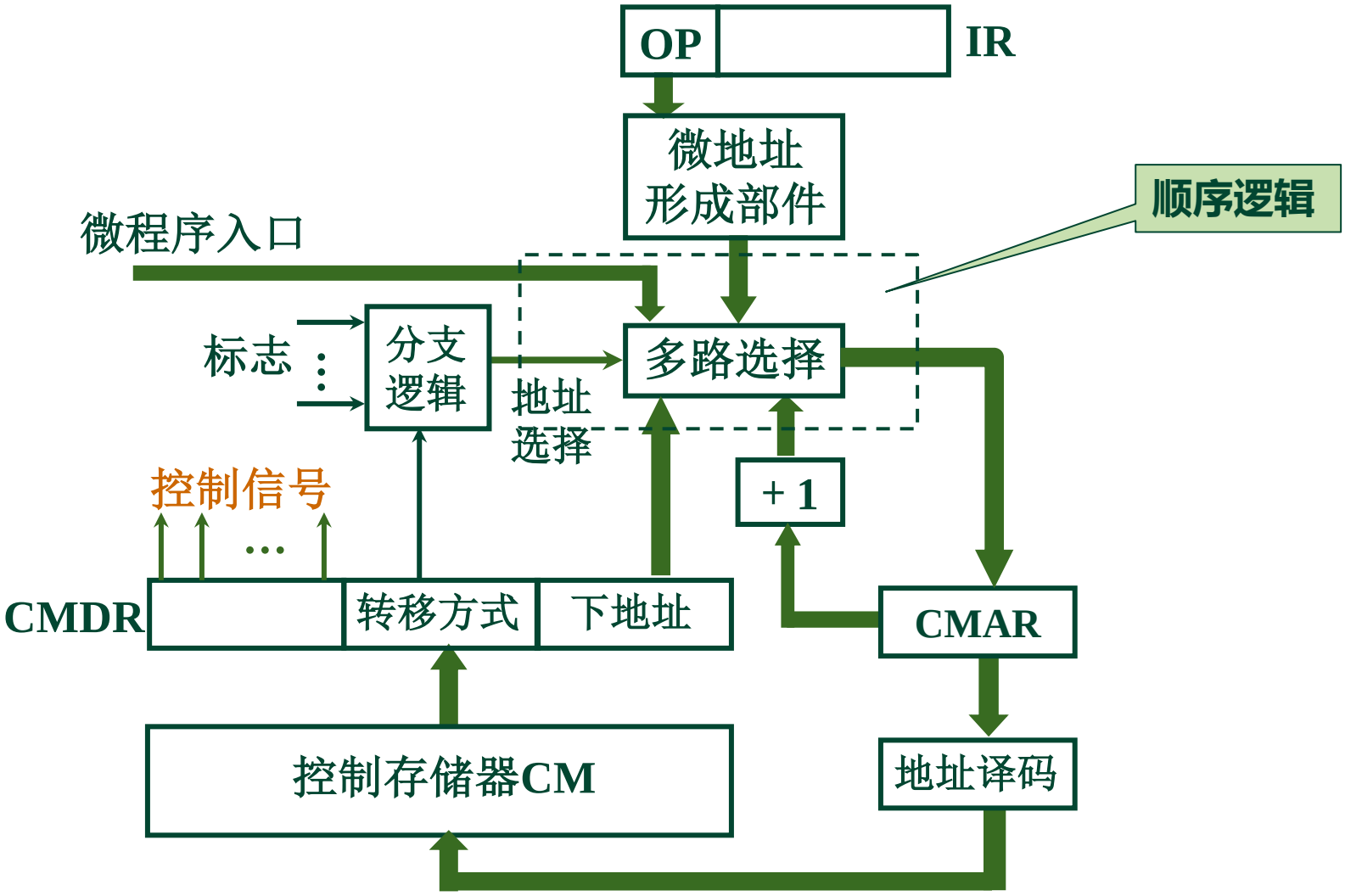
6. 由硬件产生微程序入口地址

加电后，第一条微指令地址　　由专门 硬件 产生

中断周期　　由 硬件 产生 中断周期微程序首地址

四、微指令序列地址的形成

7. 后续微指令地址形成方式原理图



五、微指令格式

1. 水平型微指令　教材P411 P407图10.7

一次能定义并执行多个并行操作

如　直接编码、字段直接编码、字段间接编码、直接和字段混合编码

2. 垂直型微指令　教材P411 表10.2

类似机器指令操作码 的方式

由微操作码字段规定微指令的功能

3. 两种微指令格式的比较

(1) 水平型微指令比垂直型微指令 并行操作能力强，灵活性强

(2) 水平型微指令执行一条机器指令所要的微指令 数目少，速度快

(3) 水平型微指令 用较短的微程序结构换取较长的微指令结构

(4) 水平型微指令与机器指令 差别大

单选题 1分

设置

【2012统考真题】某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有(　)。

A 5位

B 6位

C 15位

D 33位

提交

六、静态微程序设计和动态微程序设计

静态　微程序无须改变，采用 **ROM**

动态　通过 改变微指令 和 微程序 改变机器指令，
有利于仿真，采用 **EPROM**

七、毫微程序设计

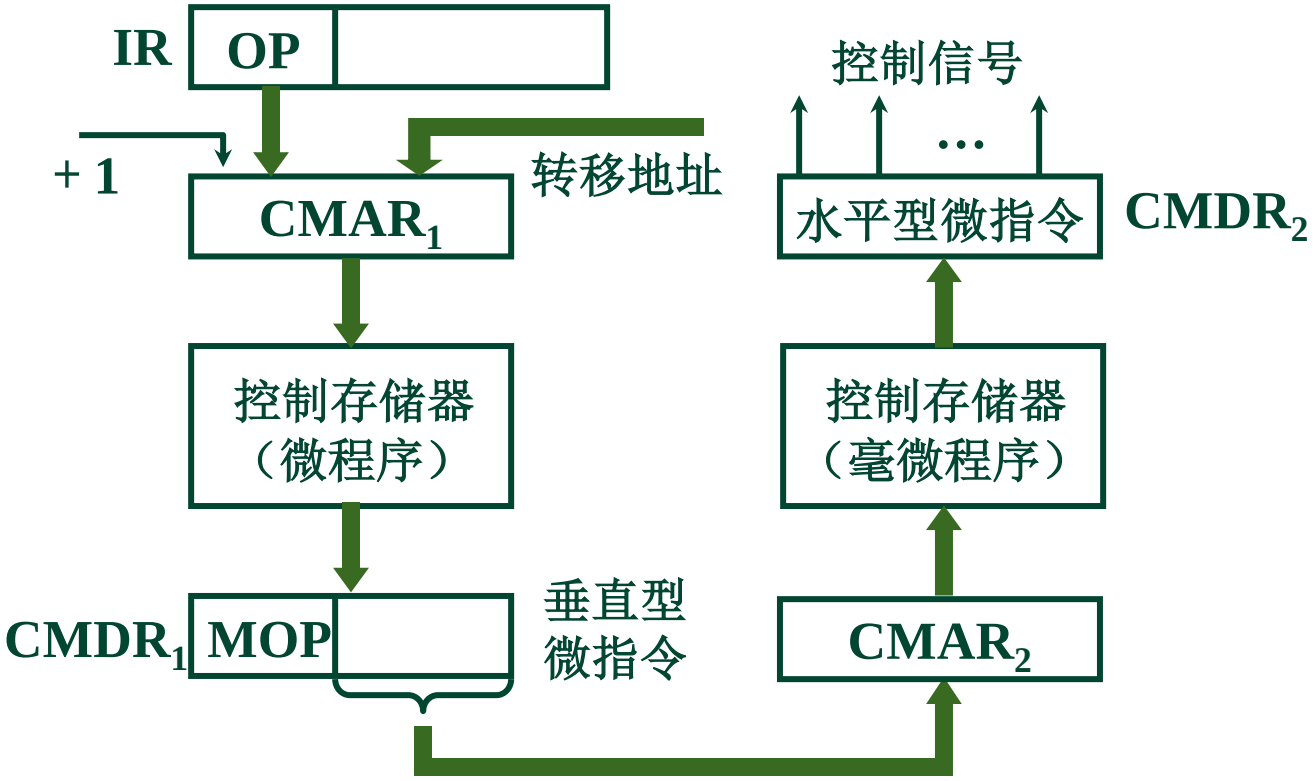
1. 毫微程序设计的基本概念

微程序设计　　用 微程序解释机器指令

毫微程序设计　用 毫微程序解释微程序

毫微指令与微指令 的关系好比 微指令与机器指令 的关系

2. 毫微程序控制存储器的基本组成



串行 微程序控制

取第 *i* 条微指令	执行第 *i* 条微指令	取第 *i*+1 条微指令	执行第 *i*+1 条微指令

并行 微程序控制

取第 *i* 条微指令	执行第 *i* 条微指令		
	取第 *i*+1 条微指令	执行第 *i*+1 条微指令	
		取第 *i*+2 条微指令	执行第 *i*+2 条微指令

九、微程序设计举例

10.2

1. 写出对应机器指令的微操作及节拍安排

假设 CPU 结构与组合逻辑相同（非总线方式）

(1) 取指阶段微操作分析　　3 条微指令

T_0　PC ⟶ MAR　　1 ⟶ R

T_1　M (MAR) ⟶ MDR　　(PC) + 1 ⟶ PC

T_2　MDR ⟶ IR　　OP (IR) ⟶ 微地址形成部件

还需考虑 如何读出 这 3 条微指令？

Ad (CMDR) ⟶ CMAR

OP (IR) ⟶ 微地址形成部件 ⟶ CMAR

教材P415

(2) 取指阶段的微操作及节拍安排

考虑到需要 形成后续微指令的地址

T_0 PC ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR　(PC)+1 ⟶ PC

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ IR　　OP (IR) ⟶ 微地址形成部件

T_5 OP (IR) ⟶ 微地址形成部件 ⟶ CMAR

(3) 执行阶段的微操作及节拍安排

考虑到需形成后续微指令的地址

取指微程序的入口地址 M
由微指令下地址字段指出

- 非访存指令

① CLA 指令

T_0　0 ⟶ AC

T_1　Ad (CMDR) ⟶ CMAR

② COM 指令

T_0　$\overline{\text{AC}}$ ⟶ AC

T_1　Ad (CMDR) ⟶ CMAR

③ SHR 指令

T_0　　L (AC) ⟶ R (AC)　　AC_0 ⟶ AC_0

T_1　　Ad (CMDR) ⟶ CMAR

④ CSL 指令

T_0　　R (AC) ⟶ L (AC)　　AC_0 ⟶ AC_n

T_1　　Ad (CMDR) ⟶ CMAR

⑤ STP 指令

T_0　　0 ⟶ G

T_1　　Ad (CMDR) ⟶ CMAR

• 访存指令

⑥ ADD 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 (AC) + (MDR) ⟶ AC

T_5 Ad (CMDR) ⟶ CMAR

⑦ STA 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ W

T_1 Ad (CMDR) ⟶ CMAR

T_2 AC ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ M (MAR)

T_5 Ad (CMDR) ⟶ CMAR

⑧ LDA 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ AC

T_5 Ad (CMDR) ⟶ CMAR

• 转移类指令

⑨ JMP 指令

T_0　　Ad (IR) ⟶ PC

T_1　　Ad (CMDR) ⟶ CMAR

⑩ BAN 指令

T_0　　$A_0 \cdot Ad(IR) + \overline{A_0} \cdot (PC) \longrightarrow PC$

T_1　　Ad (CMDR) ⟶ CMAR

全部微操作　20个

微指令　　38条

2. 确定微指令格式

(1) 微指令的编码方式

采用直接控制

(2) 后续微指令的地址形成方式

由机器指令的操作码通过微地址形成部件形成

由微指令的下地址字段直接给出

(3) 微指令字长

由 20 个微操作

确定 操作控制字段　　最少 20 位

由 38 条微指令

确定微指令的 下地址字段 为 6 位

微指令字长 可取 20 ＋ 6 ＝ 26 位

(4) 微指令字长的确定

38 条微指令中有 19 条

是用于形成后续微指令地址 ⟶ CMAR

其中 { 1 条　OP (IR) ⟶ 微地址形成部件 ⟶ CMAR
　　 { 18 条　Ad (CMDR) ⟶ CMAR

若用 Ad (CMDR) 直接送控存地址线

则 省去了输至 CMAR 的时间，省去了 CMAR　教材P417

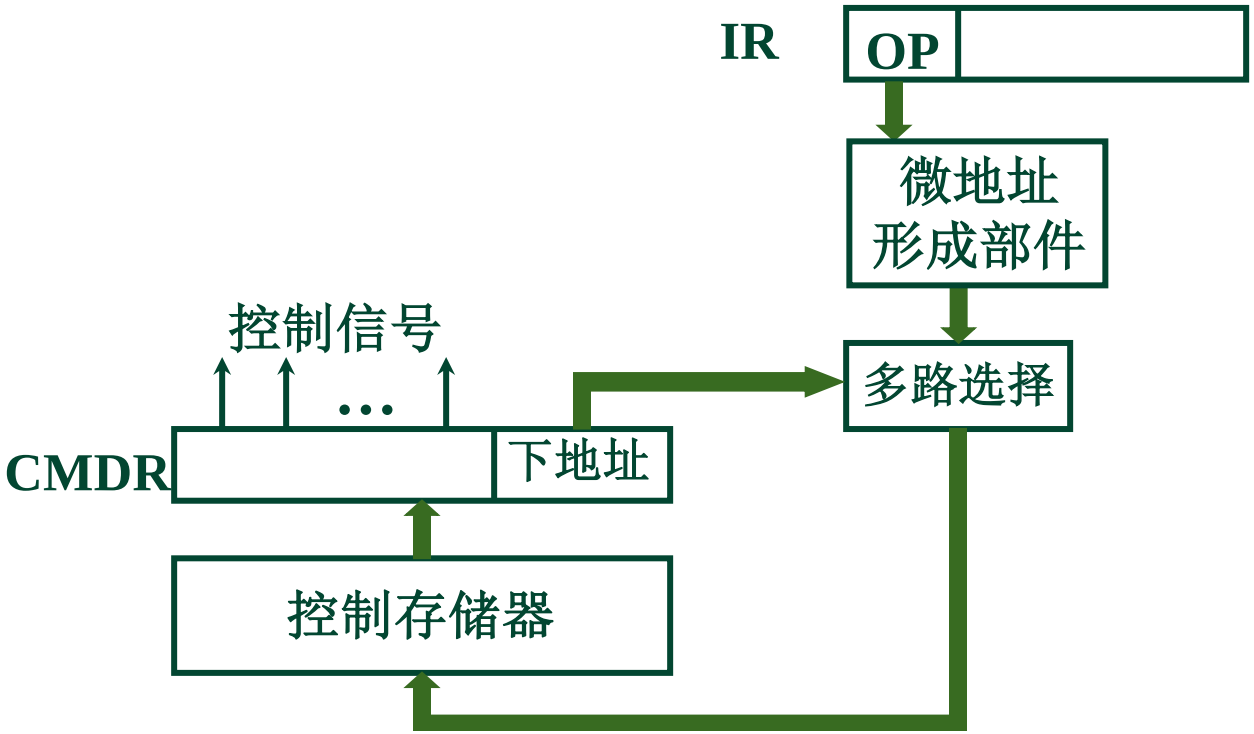
同理　OP (IR) ⟶ 微地址形成部件 ⟶ 控存地址线　教材P418 图10.16

可省去 19 条微指令，2 个微操作

38 － 19 ＝ 19　　20 － 2 ＝ 18

下地址字段最少取 5 位　　操作控制字段最少取 18 位

(5) 省去了 CMAR 的控制存储器



考虑留有一定的余量

取操作控制字段　18 位→24 位
下地址字段　5 位→ 6 位
} 共 30 位

(6) 定义微指令操作控制字段每一位的微操作



3. 编写微指令码点

微程序名称	微指令地址(八进制)	微指令（二进制代码）														
		操作控制字段									下地址字段					
		0	1	2	3	4	…	10	…	23	24	25	26	27	28	29
取指	00	1	1								0	0	0	0	0	1
	01			1	1						0	0	0		1	0
	02					1					×	×	×	×	×	×
CLA	03										0	0	0	0	0	0
COM	04										0	0	0	0	0	0
ADD	10		1					1			0	0	1	0	0	1
	11			1							0	0	1	0	1	0
	12										0	0	0	0	0	0
LDA	16		1					1			0	0	1	1	1	1
	17			1							0	1	0	0	0	0
	20										0	0	0	0	0	0

PC→MAR

M(MAR)→MDR

1→R

MDR→IR

PC+1→PC

Ad(IR)→MAR

1→R

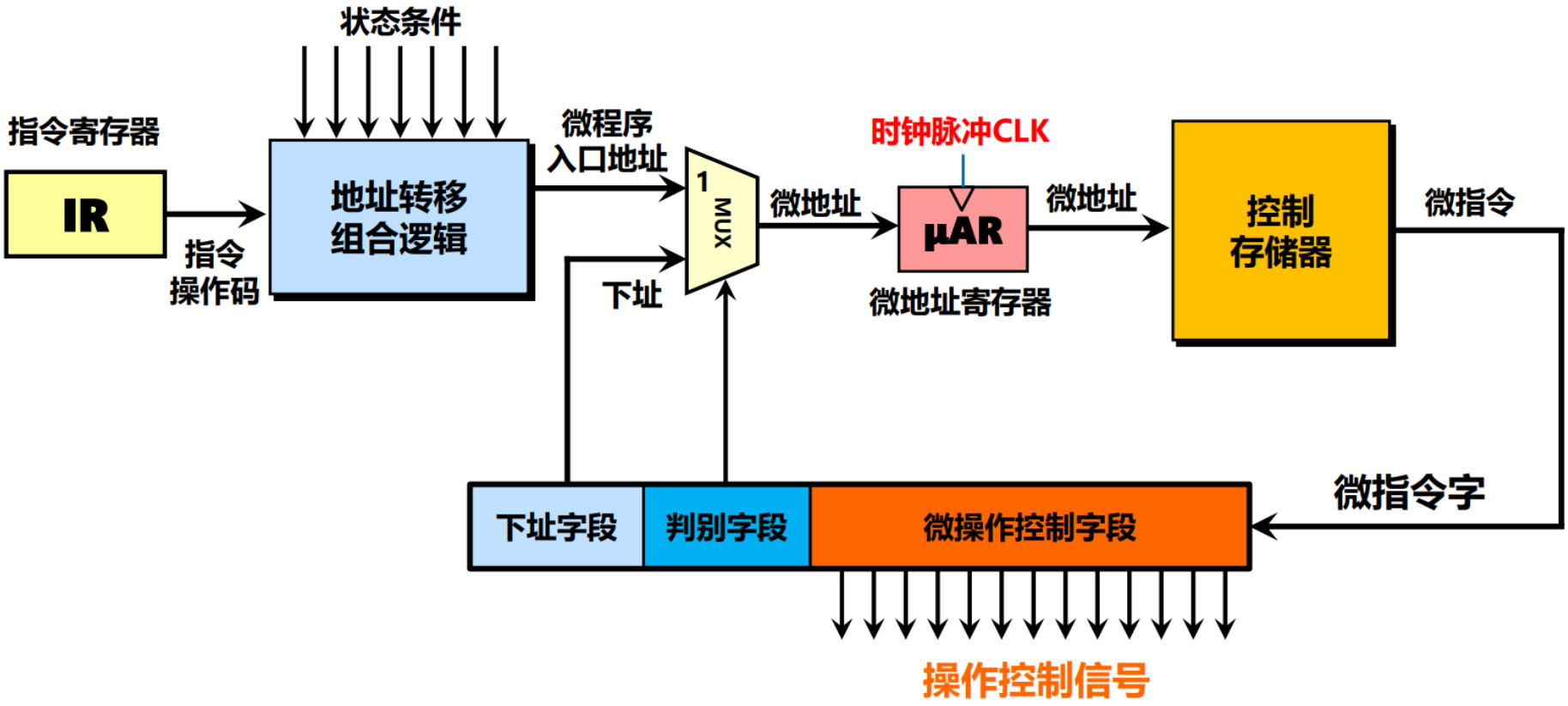
1→R

Ad(IR)→MAR

M(MAR)→MDR

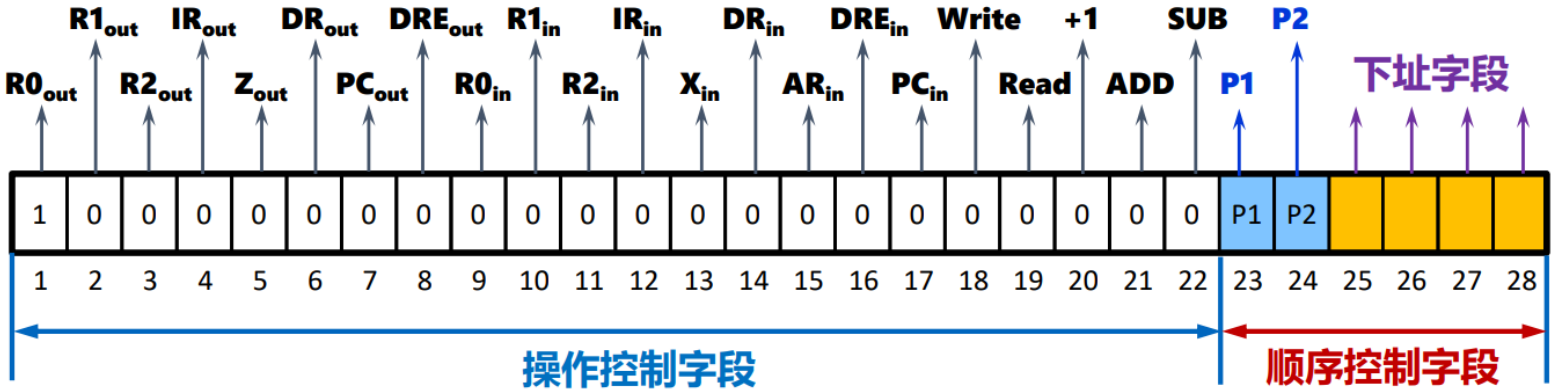
实例：微程序设计-单总线CPU*

1 微程序控制器组成原理框图



实例：微程序设计-单总线CPU

2 微指令格式



- 一条微指令对应一个时钟周期
- 微指令操作控制字段的信号在该时钟周期内有效
- 指令需要多少时钟周期就包括多少微指令

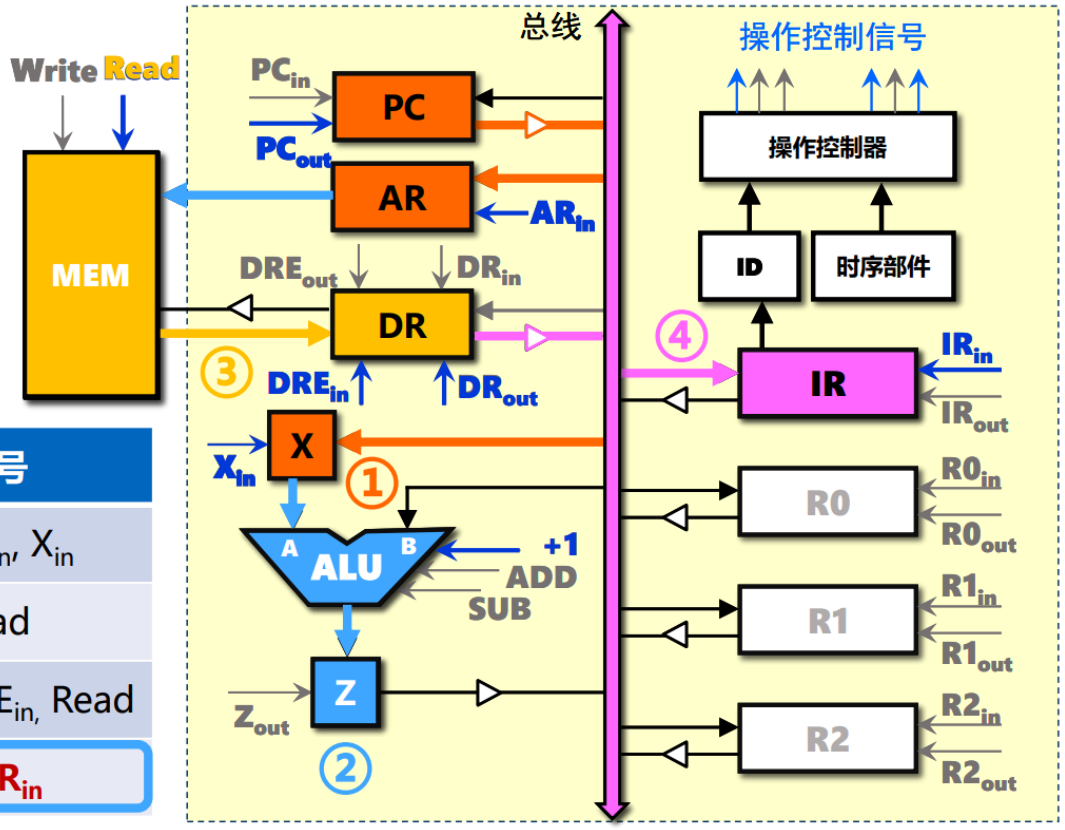
实例：微程序设计-单总线CPU

3 取指令数据通路

Mem[PC++] → IR

- **4个时钟周期**
- **四条微指令**

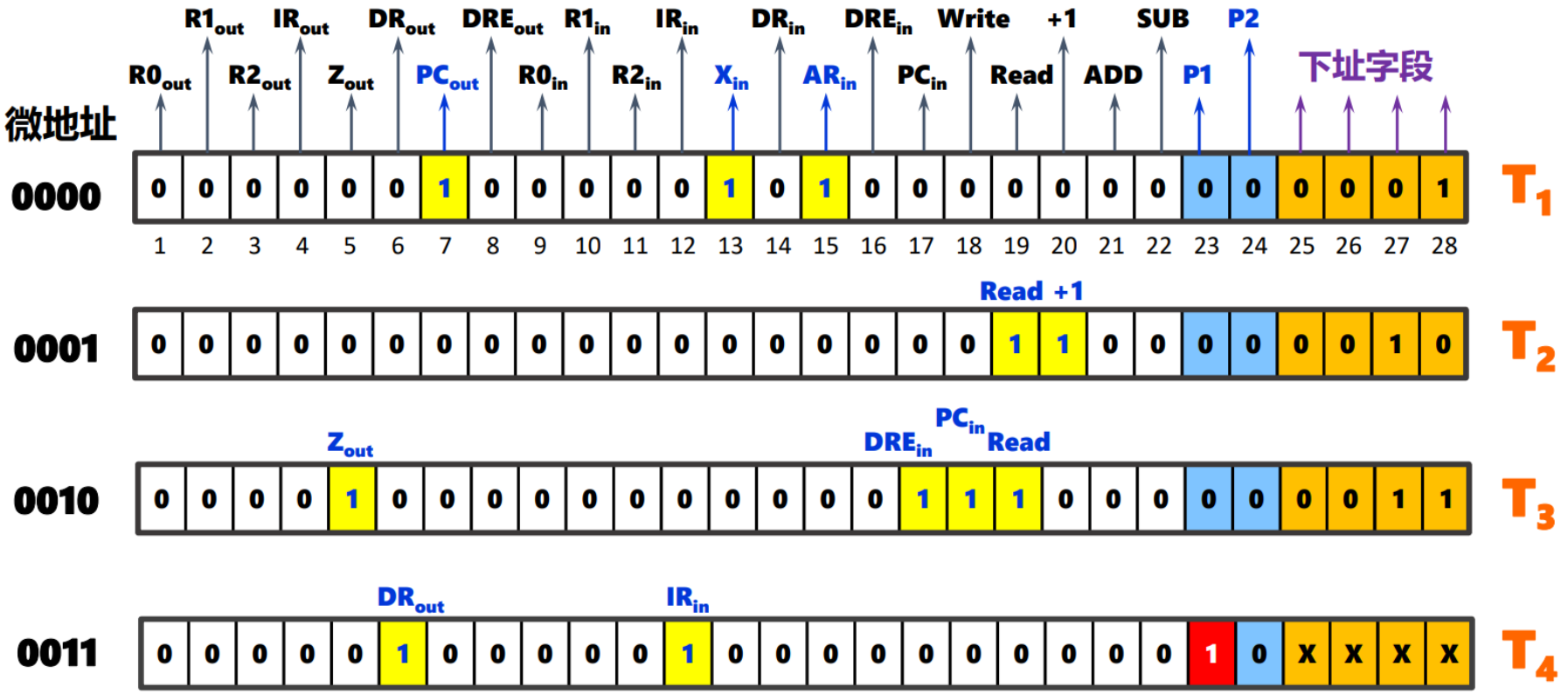
节拍	数据通路	控制信号
T1	(PC)→AR, (PC)→X	PC_{out}, AR_{in}, X_{in}
T2	(X)+1→Z	+1, Read
T3	(Z)→PC, Mem[AR]→DR	Z_{out}, PC_{in}, DRE_{in}, Read
T4	**(DR)→IR**	**DR_{out}, IR_{in}**



实例：微程序设计-单总线CPU

4 取指令微程序

节拍	取指令数据通路	控制信号
T1	(PC)→AR，(PC)→X	PC_{out}, AR_{in}, X_{in}
T2	(X)+1→Z	+1, Read
T3	(Z)→PC，Mem[AR]→DR	Z_{out}, PC_{in}, DRE_{in}, Read
T4	(DR)→IR	DR_{out}, IR_{in}



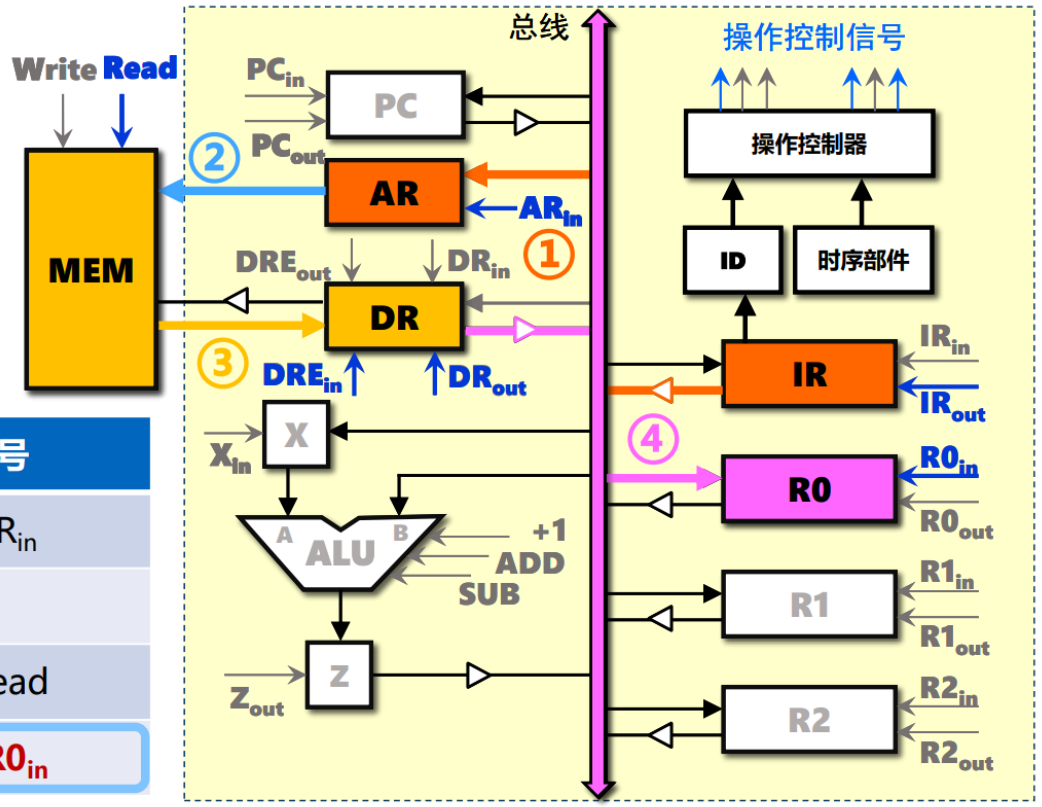
实例：微程序设计-单总线CPU

5 LOAD指令执行数据通路

LOAD R0,6#

Mem[IR_A] → Reg

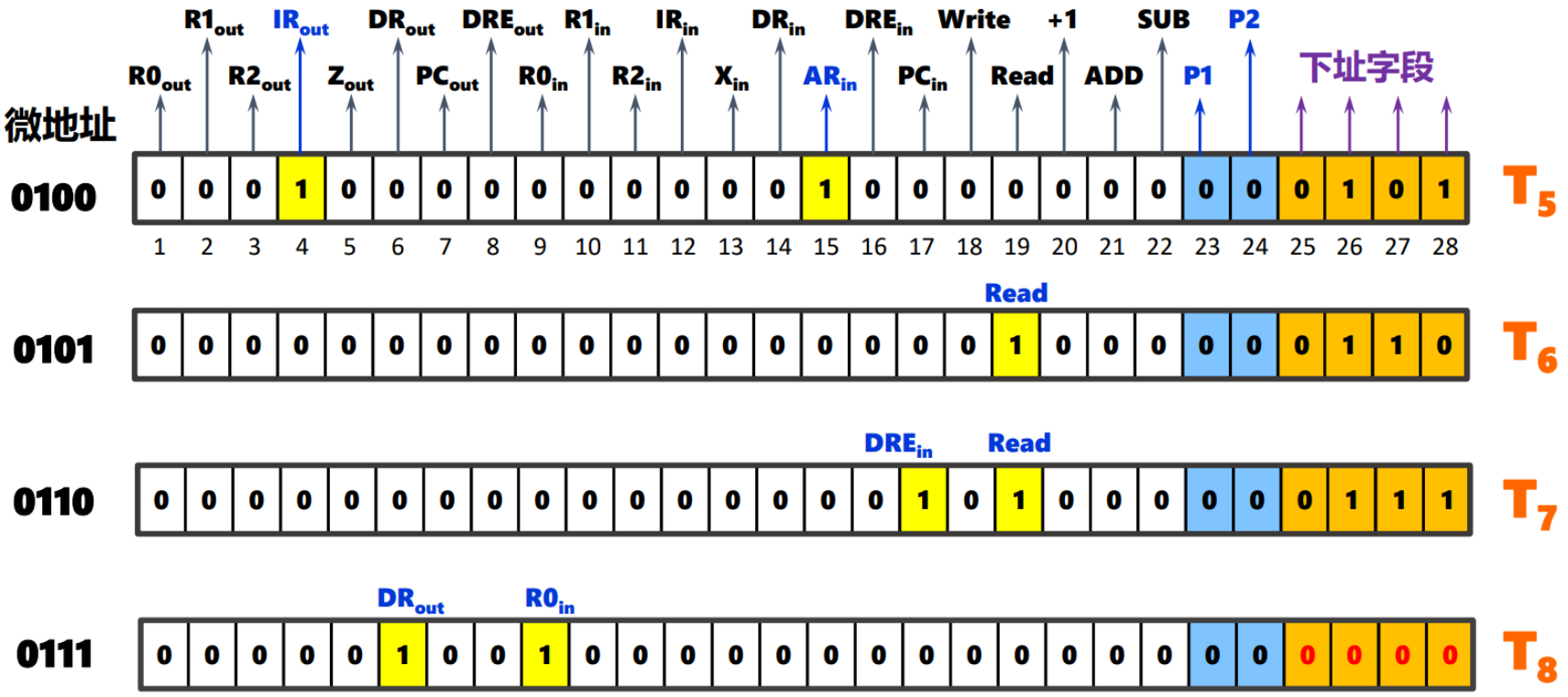
节拍	数据通路	控制信号
T1	(IR_A)→AR	IR_{out}, AR_{in}
T2		Read
T3	Mem[AR]→DR	DRE_{in},Read
T4	**(DR)→R0**	**DR_{out}, $R0_{in}$**



实例：微程序设计-单总线CPU

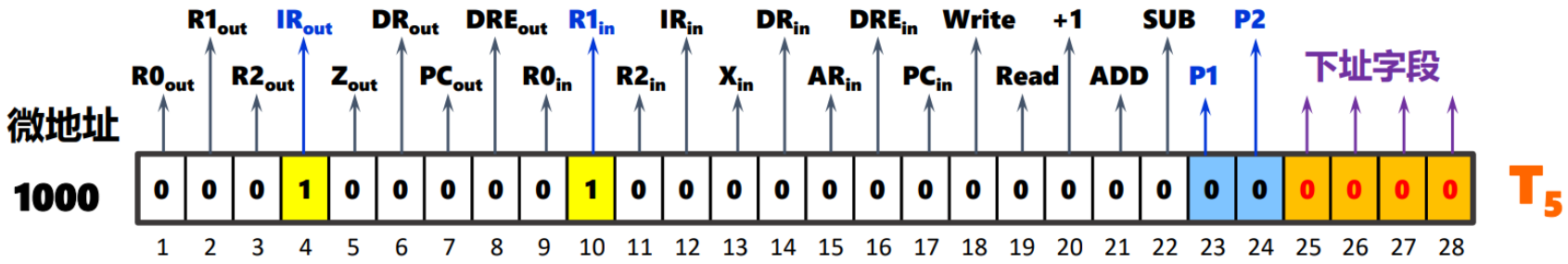
6 LOAD指令微程序

节拍	取指令数据通路	控制信号
T1	$(IR_A)\rightarrow AR$	IR_{out}, AR_{in}
T2		Read
T3	Mem[AR]→DR	DRE_{in},Read
T4	(DR)→R0	DR_{out}, $R0_{in}$



实例：微程序设计-单总线CPU

7 MOVE指令微程序

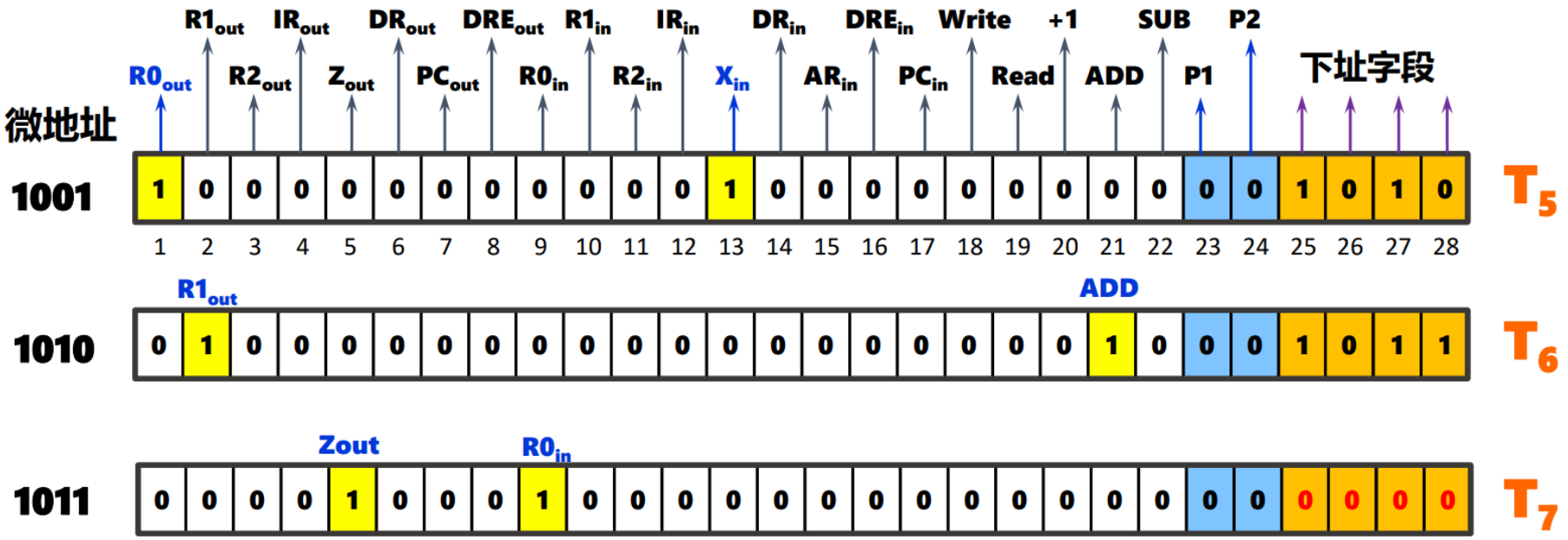


节拍	数据通路	控制信号
T1	$(IR_A) \rightarrow R[0]$	IR_{out}, $R1_{in}$
T2		
T3		
T4		

实例：微程序设计-单总线CPU

节拍	数据通路	控制信号
T1	(R0)→X	$R0_{out}$, X_{in}
T2	(X)+(R1)→Z	$R1_{out}$,ADD
T3	(Z)→R0	Z_{out},$R0_{in}$
T4		

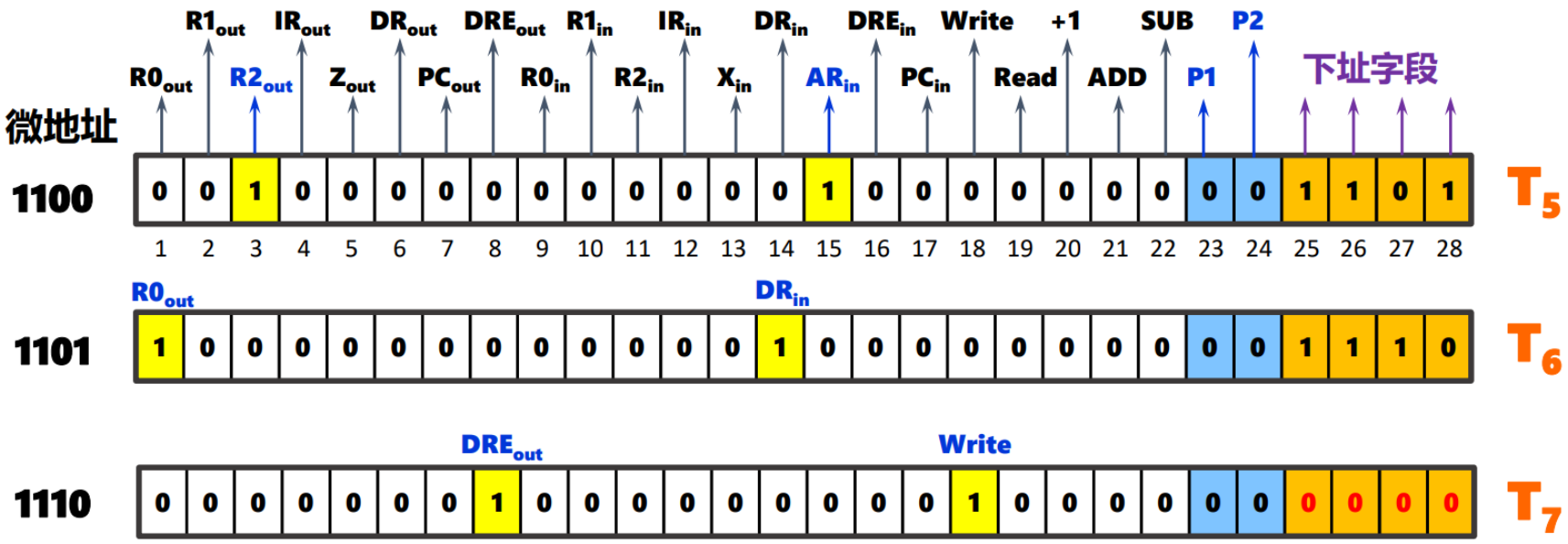
8 ADD指令微程序



实例：微程序设计-单总线CPU

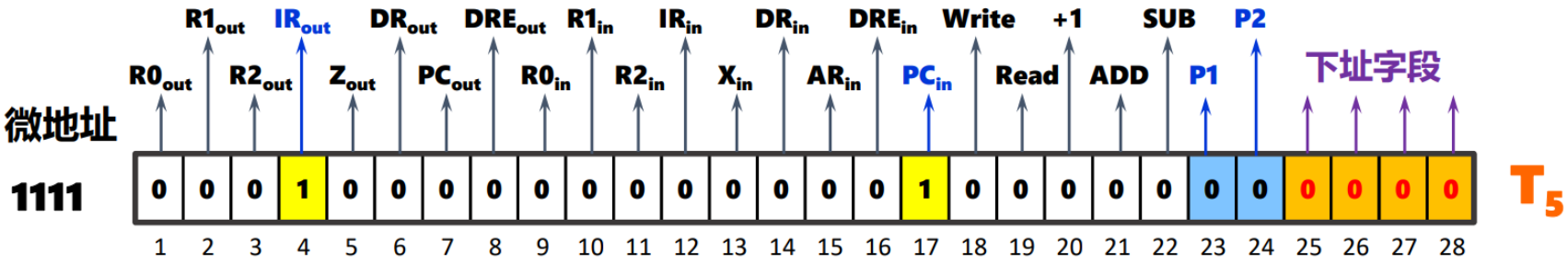
9 STORE指令微程序

节拍	数据通路	控制信号
T1	(R2)→AR	$R2_{out}$, AR_{in}
T2	(R0)→DR	$R0_{out}$, DR_{in}
T3	（DR）→Mem[AR]	DRE_{out}, Write
T4		



实例：微程序设计-单总线CPU

A JMP指令微程序



节拍	数据通路	控制信号
T1	$(IR_A) \rightarrow PC$	IR_{out}, PC_{in}
T2		
T3		
T4		

实例：微程序设计-单总线CPU

B 单总线CPU微程序

微地址	操作控制字段																						顺序控制字段						
	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)	P(15)	P(16)	P(17)	P(18)	P(19)	P(20)	P(21)	P(22)	S1	S2	S3	S4	S5	S6	
0000	0	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	取指令微程序
0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	
0010	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	1	1	
0011	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	X	X	X	X	
0100	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	LOAD微程序
0101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	0	
0110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	1	1	1	
0111	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1000	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	MOVE微程序
1001	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	ADD 微程序
1010	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	1	
1011	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1100	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	1	0	1	STORE微程序
1101	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0	
1110	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
1111	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	JMP 微程序

单选题 1分

设置

【2009统考真题】相对于微程序控制器，硬布线控制器的特点是(　)。

A 指令执行速度慢，指令功能的修改和扩展容易

B 指令执行速度慢，指令功能的修改和扩展难

C 指令执行速度快，指令功能的修改和扩展容易

D 指令执行速度快，指令功能的修改和扩展难

提交





Computer Organization

Thank You !

Computer Architecture Research Institute · Computer Organization

期末考试题型

- 一、选择题（15题，30分）
- 二、填空题（10空，10分）
- 三、判断题（ 10题，10分）
- 四、简答题（3题，15分）
- 五、计算题（1题，10分）
- 六、设计题（2题，15+10=25分）

举例（小数为例）

- 例1：真值 $x = -0.10001$，求$[x]_{补}$

解：$[x]_{原} = 1.10001$，$[x]_{反} = 1.01110$

$[x]_{补} = [x]_{反} + 2^{-5} = 1.01110 + 0.00001 = 1.01111$

小技巧：$x = -0.10001$ ← 从后往前找到第一个1

原则：第一个1不变，前面包含符号位所有取反，负号去掉，即得结果。

$[x]_{补} = 1.01111$

- 例2：真值 $x = -0.10100$，求$[x]_{补}$
- 例3：真值 $x = -0.10000$，求$[x]_{补}$
- 例4：$[x]_{补} = 0.10010$，求$[-x]_{补}$
- 例5：$[x]_{补} = 1.10000$，求$[-x]_{补}$

85

10.3 多处理器基本概念

一、SISD、SIMD、 MISD 、MIMD、向量处理器的基本概念

- 费林分类法（Flynn's Taxonomy），是一种高效能计算机的分类方式。

1972年费林（Michael J. Flynn）根据资讯流（Information Stream）可分成指令（Instruction）和数据（Data）两种。

1、SISD（ Single instruction, Single Data ）单指令流单数据流

2、SIMD （ Single instruction, Multiple Data ）单指令流多数据流

3、MISD （ Multiple instruction Single Data ）多指令流单数据流

4、MIMD （ Multiple instruction, Multiple Data ）多指令流多数据流

1、SISD（ Single instruction, Single Data ）单指令流单数据流

传统的串行计算机

特性：指令序列只能并行，不能并发；每条指令只能处理1-2个数据；不是数据级并行数据；

硬件：1个处理器 + 1个主存

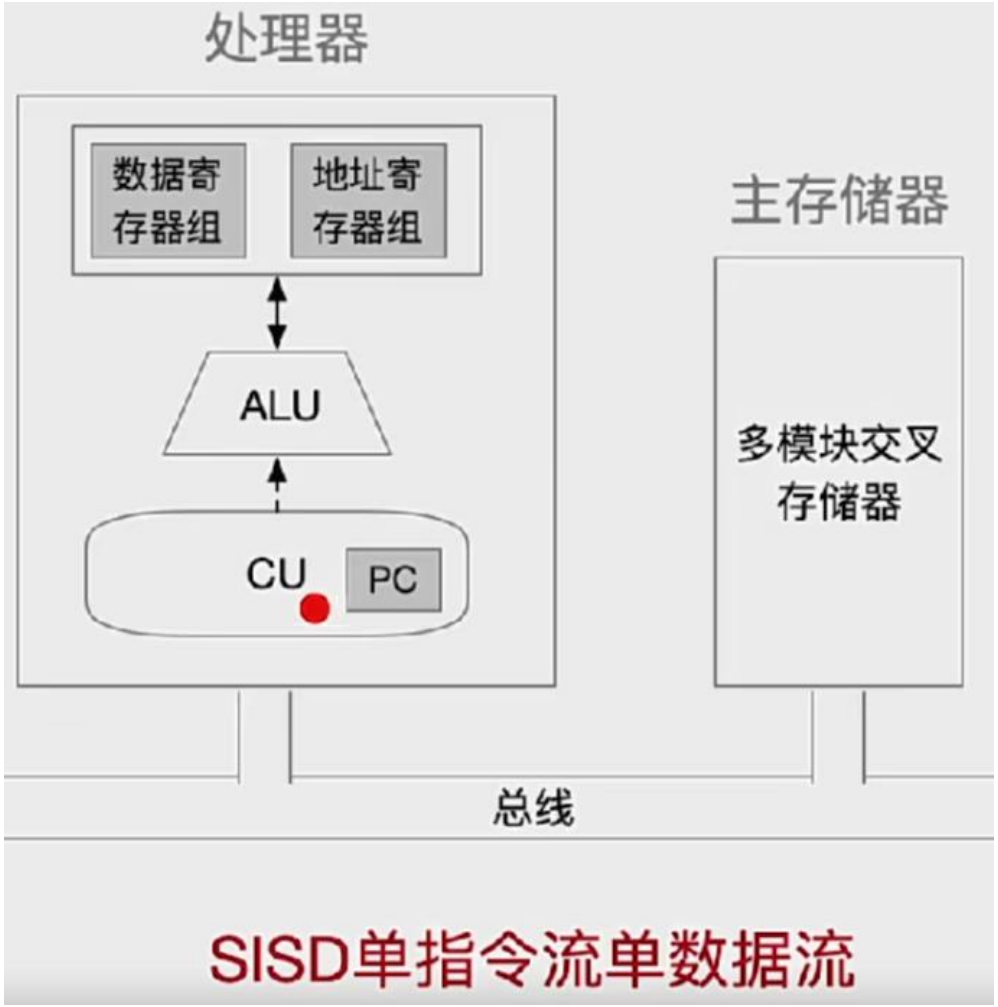
提高速度策略：

（1）采用流水线

（2）设置多个功能部件

（3）采用多模块交叉存储器

（内存的低位交叉编址）



经典冯·诺依曼架构

2、SIMD （ Single instruction, Multiple Data ）单指令流多数据流

- 特性：各指令序列只能并发不能并行；每条指令可以同时处理多个具有相同特征的数据；是一种实现数据级并行的技术；

- 硬件：

1个控制器（CU）+ 多个运算器（ALU） + 多个局存储器 + 1个主存

每个ALU都有各自的寄存器堆、局部存储器、地址寄存器

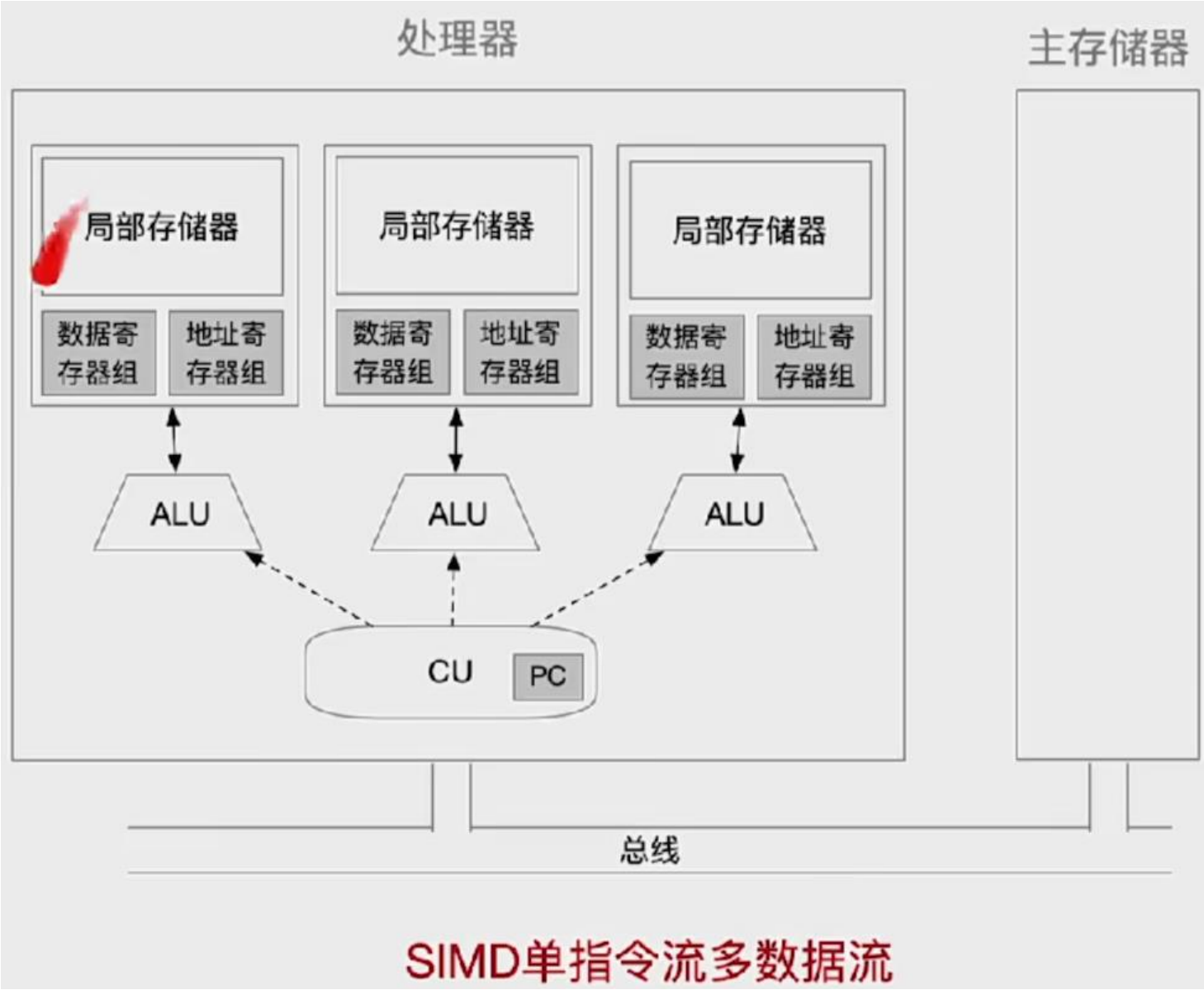
不同的ALU执行同一条指令，处理不同数据。

整机只有一个PC

- 应用场景：GPU/多媒体指令集(SSE/AVX)；优化for循环中数组元素的重复处理；

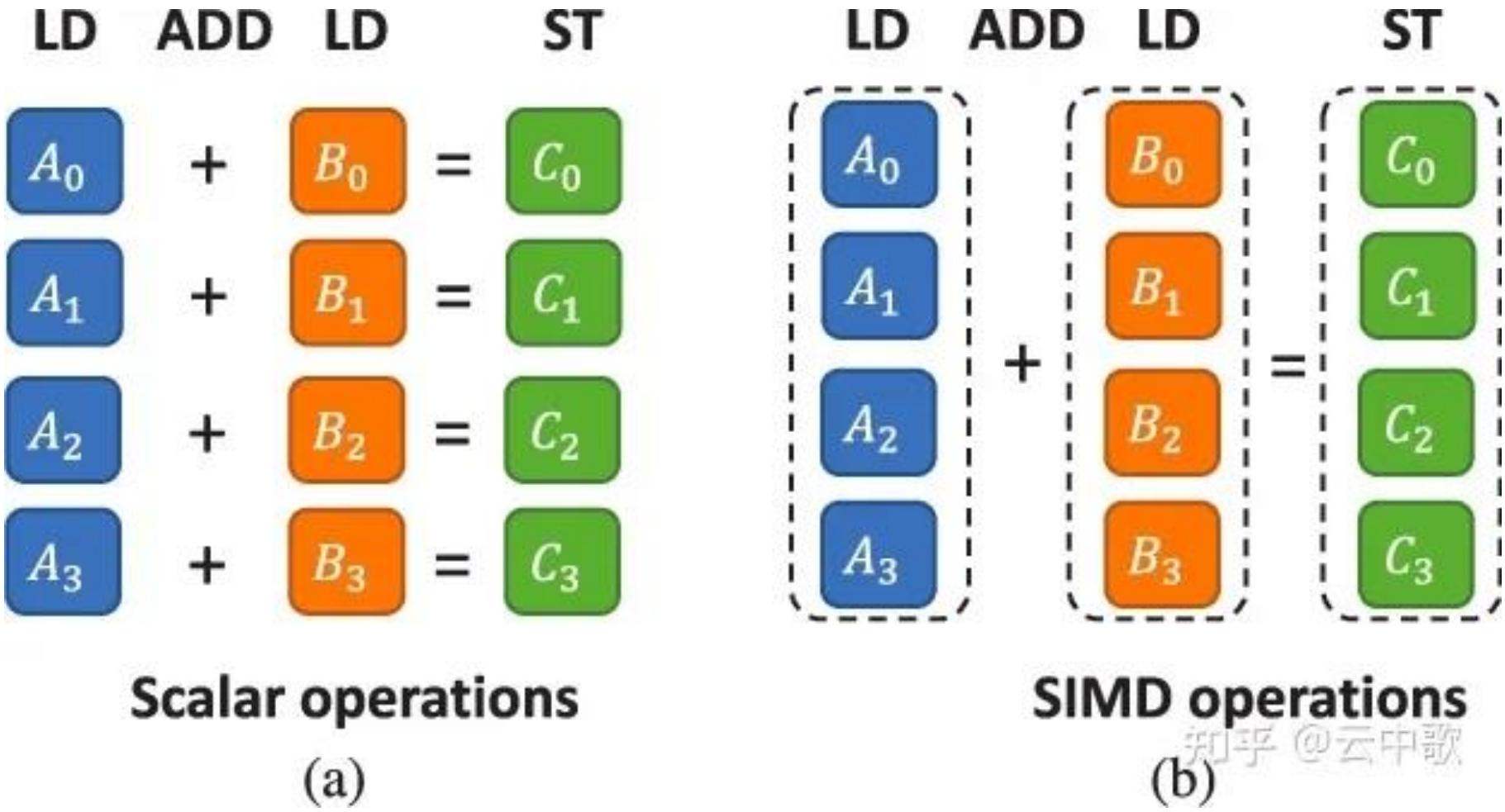
- 其典型代表是向量处理器（Vector Processor）和阵列处理器（Array Processor）。见10.3

一、SISD、SIMD、 MISD 、MIMD、向量处理器的基本概念



SIMD单指令流多数据流

一、SISD、SIMD、 MISD 、MIMD、向量处理器的基本概念



知乎 @云中歌

以加法指令为例，在SISD架构计算机上，CPU先执行一条指令，进行A0 + B0 = C0计算，再执行下一条指令，进行A1 + B1 = C1计算，按此顺序依次完成后续计算。四个加法计算需依次串行执行四次。而对于SIMD指令来说，CPU只需执行一条指令，即可完成四个加法计算操作，四个加法计算操作并行执行。

10.3 多处理器基本概念

- 并行技术可以分为多种类型：

包括指令级并行、数据级并行、线程级并行和请求级并行等。

10.3 多处理器基本概念

- 从计算机系统内部程序的执行角度，将并行性从低到高划分为四级：

①指令内部并行；②指令间并行；③任务和进程并行；④作业和程序并行

- 按信息处理的步骤和阶段，并行性可分为：

①存储器操作并行；②处理器操作步骤并行；③处理器操作并行；④指令、任务、作业并行

- 实行并行处理的技术途径主要有：

①时间重叠，又称为时间并行；②资源重复，又称空间并行；③资源共享

- 针对数字图像的特点，图像处理中的并行性可分为下列四种类型：

①操作并行；②图像并行；③邻域并行；④像素位并行

3、MISD （ Multiple instruction Single Data ）多指令流单数据流

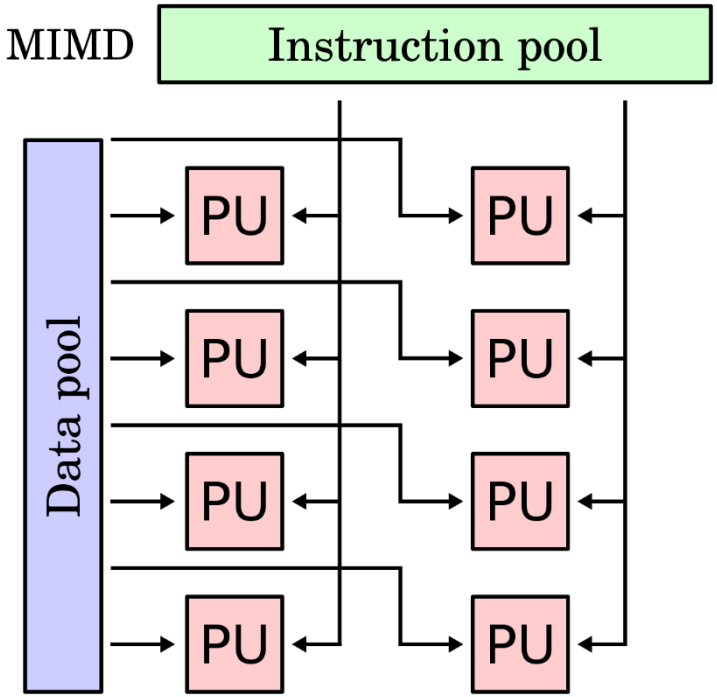
一般不用这个架构，现实中不存在这样的计算机。

因为单个数据用1个CU处理足够，没必要用多个CU。

4、MIMD （ Multiple instruction, Multiple Data ）多指令流多数据流

特性：各指令序列并行执行，分别处理多个不同的数据；
是一种线程级或线程级以上的并行技术。

现代计算机多采用此策略；



4、MIMD （ Multiple instruction, Multiple Data ）多指令流多数据流

进一步分类：

- 多处理器系统（共享存储多处理器Shared Memory multiprocessor,SMP）

硬件组成：

一台计算机内，包含多个处理器+1个主存空间

多个处理器共享单一的物理地址空间

各处理器之间，可以通过LOAD/STORE指令，访问共享的主存储器，可以通过共享的主存储器相互传送数据。

- 多计算机系统

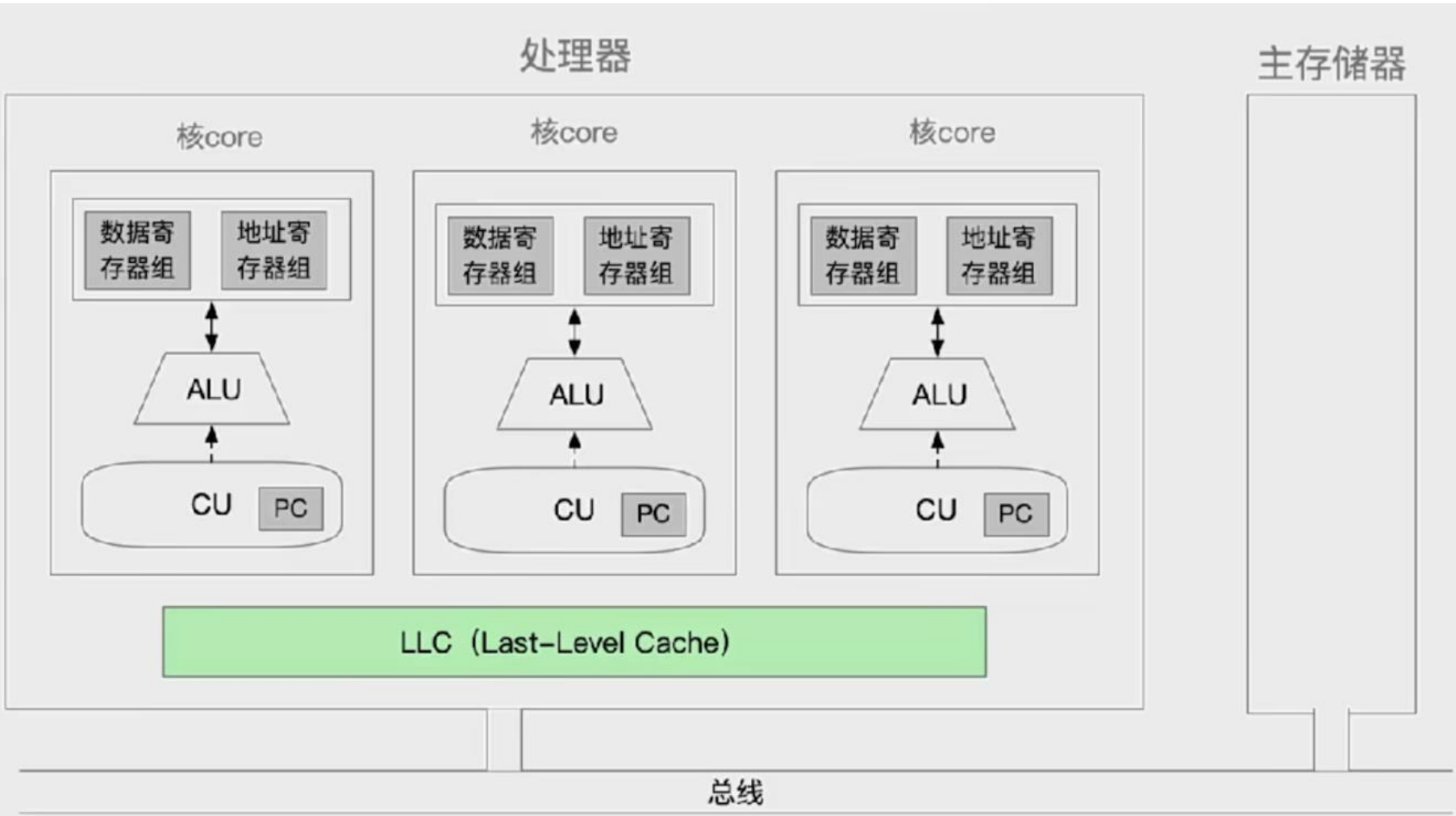
硬件组成：

由多台计算机组成，因此有多个处理器 + 多个主存储器

每台计算机拥有各种的私有查重期，物理空间相互独立。

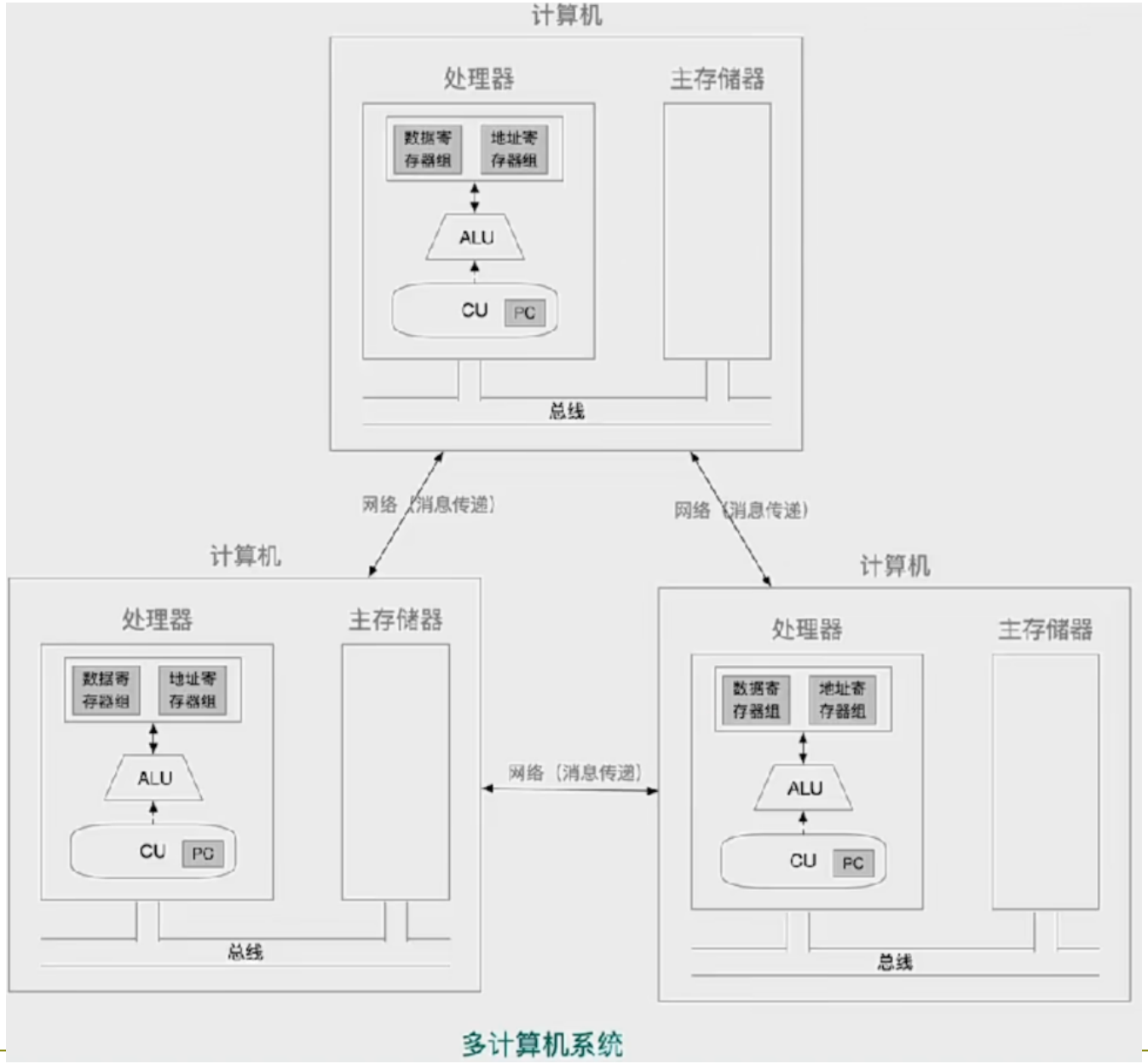
各计算机之间，不会通过LOAD/STORE指令直接访问对方的存储器，只能通过“消息传递”相互传送数据。

4、MIMD （ Multiple instruction, Multiple Data ）多指令流多数据流



共享存储多处理器（SMP）系统

4、MIMD （ Multiple instruction, Multiple Data ）多指令流多数据流



多计算机系统

二、共享内存多处理器(SMP)的基本概念

1、共享内存多处理器的基本概念和特点

- 定义：共享内存多处理器（Shared Memory Multiprocessor），是指在多处理器的计算机系统中，多个中央处理器（CPU）可以共享大容量内存的一种架构。这种架构使得多个处理器能够高效地访问和修改同一内存空间中的数据，从而实现并行处理和任务协同。

- 特点：共享内存多处理器系统通常具有高速的总线或网络连接，以确保处理器之间能够快速地交换数据。此外，该系统还需要具备高效的内存管理机制，以确保数据的一致性和完整性。

二、共享内存多处理器(SMP)的基本概念

2、共享内存多处理器的类型

- 统一存储访问多处理器（UMA）：在这种类型中，所有处理器对所有存储单元的访问时间大致相同。这意味着每个处理器都可以平等地访问内存中的任何位置，从而实现高效的数据共享。

- 非统一存储访问多处理器（NUMA）：与UMA不同，NUMA将主存划分为不同的区域，并分配给不同的处理器。处理器访问自身附近的内存（本地内存）时，速度更快、成本更低。然而，当处理器需要访问远程内存时，性能可能会受到影响。

二、共享内存多处理器(SMP)的基本概念

4、共享内存的工作原理与优势

- 工作原理：共享内存的工作原理基于操作系统的内存管理机制。当多个处理器需要访问同一数据时，操作系统会将这些数据映射到共享内存空间中。这样，每个处理器都可以通过访问共享内存来读取或修改数据，而无需通过缓慢的总线或网络进行数据传输。
- 优势：
  - 提高性能：共享内存允许多个处理器同时访问和修改同一数据，从而显著提高了系统的并行处理能力和整体性能。
  - 降低开销：通过减少数据传输和同步的开销，共享内存能够降低系统的整体功耗和延迟。
  - 简化编程：共享内存为程序员提供了一种简单而直接的数据共享方式，无需编写复杂的通信和同步代码。

二、共享内存多处理器(SMP)的基本概念

5、共享内存多处理器的应用场景

- 高性能计算：在科学计算、工程模拟、数据分析等高性能计算领域，共享内存多处理器能够显著提高计算速度和精度。
- 实时系统：在实时控制系统、航空航天、军事等领域，共享内存多处理器能够提供低延迟、高可靠性的数据处理能力。
- 分布式系统：在分布式计算环境中，共享内存技术可以用于实现高效的节点间通信和数据共享，从而提高系统的可扩展性和性能。

综上所述，共享内存多处理器是一种高效、可靠、灵活的计算机架构，适用于各种高性能计算和实时处理场景。

三、多核处理器（multi-core）的基本概念

1、多核处理器（ Multi-core processor ）的定义

- 基本概念：多核处理器是指在一枚处理器中集成两个或多个完整的计算引擎（即内核）。这些内核能够支持系统总线上的多个处理器操作，由总线控制器统一提供所有总线控制信号和命令信号。
- 定义解读：多核处理器是处理器技术的一大进步，它通过集成多个计算内核，实现了处理器性能的显著提升。每个内核都能独立执行指令和处理数据，从而大幅提高了处理器的并行处理能力。

三、多核处理器（multi-core）的基本概念

2、多核处理器的技术原理与架构

- 技术原理：多核处理器的技术原理在于“分治法”战略，即将复杂的计算任务划分为多个子任务，然后分配给不同的处理内核进行并行处理。这种并行处理的方式能够显著提高计算效率，缩短计算时间。
- 架构特点：多核处理器的架构特点是集成度高、并行处理能力强。它采用，单将枚每个芯片执行（内核硅作为核分）立的设计逻辑，能够直接插入单一的处理器插槽中。操作系统会利用所有相关的资源处理器进行管理和调度。

三、多核处理器（multi-core）的基本概念

- 多核处理器的应用领域
  - 服务器：多核处理器在服务器领域得到了广泛应用。服务器通常需要处理大量的并发请求和数据传输任务，多核处理器能够提供强大的处理能力，满足这些需求。
  - 高性能计算：在科学计算、模拟和大规模数据分析等任务中，多核处理器能够同时处理多个数据块或执行多个计算任务，大幅提高机器的整体计算性能。
  - 嵌入式系统：多核处理器还支持多种节能模式，使得其在移动设备和嵌入式系统中得到了广泛应用。这些系统通常对功耗和性能有较高要求，多核处理器能够很好地满足这些需求。
  - 虚拟化与云计算：在虚拟化和云计算环境中，多核处理器能够同时运行多个虚拟机或容器，提高了整体系统的效率。这使得多核处理器成为虚拟化技术和云计算技术的重要支撑。

三、多核处理器（multi-core）的基本概念

- 多核处理器的优势与发展
  - 优势：
    - *性能提升*：多核处理器能够同时执行多个线程或进程，显著提高了计算性能。在科学计算、图像处理、视频渲染等需要大量计算资源的领域，多核处理器能够显著缩短计算时间。
    - *能效比高*：多核处理器通过优化核心架构和动态电源管理技术，实现了高性能与低功耗的平衡。这使得多核处理器在保持高性能的同时，能够显著降低功耗和热量产生。
    - *多任务并行*：多核处理器能够将不同的任务分配给不同的核心进行处理，提高了系统的多任务并行执行能力。这使得用户可以同时运行多个应用程序，而不会出现卡顿或延迟现象。
  - 发展：随着半导体工艺技术的不断进步和体系结构的不断发展，多核处理器的性能将持续提升。未来，多核处理器将更加注重能效比的优化和并行编程框架的支持，以满足更加复杂和多样化的应用场景需求。

三、多核处理器（multi-core）的基本概念

5、多核处理器与单核处理器的优势对比是什么

- 多核处理器的优势：
  - 能同时处理多个任务，提高系统整体性能。
  - 通过并行处理，可以缩短复杂计算任务的执行时间。
  - 在多任务环境下，能够提高系统的响应速度和流畅度。
  - 更易于实现功耗的优化，提高能效比。
- 单核处理器的优势（相对而言较少，且多已被多核技术超越）：
  - 结构相对简单，可能在一些特定应用场景下成本更低。
  - 对于不需要并行处理的任务，单核处理器可能足够使用，且软件兼容性较好（尽管现代软件大多已支持多核）。
- 但需要注意的是，随着技术的发展，单核处理器的性能提升已逐渐遇到瓶颈，而多核处理器已成为主流趋势，特别是在需要高性能计算和多任务处理的场景中

四、向量处理器（SIMD思想的进阶）

向量处理器是SIMD的变体，是一种实现了直接操作一维数组(向量)指令集的CPU，而串行处理器只能处理单一数据集。

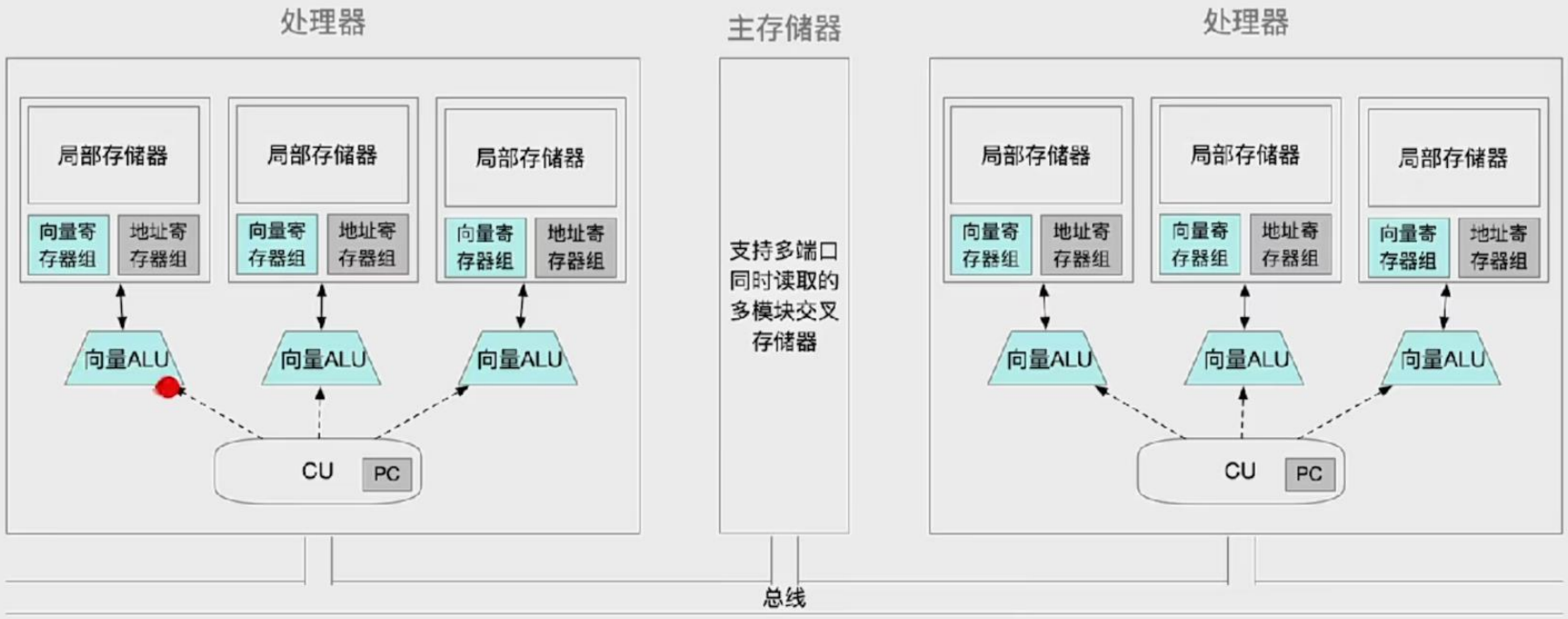
其特性为: 一条指令的处理对象是“向量”，擅长对向量型数据并行计算、浮点数运算, 常被用于超级计算机中， 处理科学研究中巨大运算量。

硬件组成:

多个处理单元，多组“向量寄存器”。

主存储器应采用“多个端口同时读取”的交叉多模块存储器。主存储器大小限定了机器的解题规模，因此要有大容量的、集中式的主存储器。

四、向量处理器（SIMD思想的进阶用



向量处理器

eg：向量处理机的LOAD指令，可以将一个向量取到向量寄存器中；加法指令，可以实现两个向量相加

应用于：向量计算、大量浮点数计算，空气动力学、核物理学、巨型矩阵计算问题

很多超级计算机如中国的“银河” 就是向量处理器

五、SIMT（单指令多线程，SIMD的扩展）

由GPU（如NVIDIA CUDA）实现。

单条指令控制多个线程，线程可独立处理分支（通过掩码解决分支发散）。

- 关键区别：

每个线程有独立的程序计数器和寄存器（SIMD无此特性）；

允许部分线程执行不同操作（如if-else分支）。

10.3 多处理器基本概念

应用选择：

- 规则数据并行（如图像滤波）→ SIMD。
- 复杂并行任务（如深度学习）→ SIMT（GPU）。
- 多任务异步处理（如Web服务器）→ MIMD。
- 高可靠性需求（如航天控制）→ MISD冗余设计。
- 科学计算（向量计算） → 向量处理器

技术对比矩阵

架构类型	数据流	控制流	典型应用场景
SISD	单	单	传统串行计算
SIMD	多	单	多媒体处理
MIMD	多	多	通用并行计算
向量处理	向量化	单	科学计算

小结

- SISD：单指令流单数据流，不是数据级并行技术
- SIMD：单指令流多数据流，是一种数据级并行技术，应用：向量处理机
- MISD：多指令流单数据流，现实中不存在这种计算机
- MIMD：多指令流多数据流，是一种线程级并行、甚至是线程级以上并行技术；进一步分类：
- 多处理器系统：一个CPU芯片多个核，两种命名角度：共享内存多处理器（SMP），多核处理器（multi-core）
- 多计算机系统：如：多台计算机构成的分布式系统，只能通过“消息传递”相互传送数据

六、硬件多线程的基本概念

- 1、定义：硬件多线程（Hardware Multithreading，简称HMT）是指处理器在同一时间能够处理多个线程的技术。

这些线程在硬件层面上以并行的方式执行，以提高处理器的效率和吞吐量。

- 目的：硬件多线程技术的主要目的是提高处理器的利用率，尤其是在处理那些需要等待外部资源（如内存访问、I/O操作）的线程时。通过同时运行多个线程，处理器可以在一个线程等待时切换到另一个线程，从而避免浪费计算资源。

六、硬件多线程的基本概念

- 2、硬件多线程的两种主要类型：
  - 细粒度多线程：细粒度多线程技术会在每个时钟周期或多个时钟周期内频繁地切换线程。这种方法的优点是能够更有效地利用处理器的执行单元，因为当某个线程因为某些原因（如Cache缺失）而暂停执行时，处理器可以立即切换到另一个线程继续执行。然而，频繁的线程切换也会带来一定的开销，包括保存和恢复线程上下文所需的时间和资源。
  - 粗粒度多线程：与细粒度多线程相比，粗粒度多线程技术会在较长的时间段内（如连续多个时钟周期）运行同一个线程的指令序列。这种方法的优点是减少了线程切换的开销，因为线程切换的频率相对较低。然而，粗粒度多线程可能无法像细粒度多线程那样充分地利用处理器的执行单元，特别是在处理那些包含大量等待时间的线程时。

六、硬件多线程的基本概念

3、硬件多线程的工作原理

- 线程切换：硬件多线程技术通过在线程之间快速切换来实现并行执行。当某个线程因为等待外部资源而暂停执行时，处理器会保存该线程的上下文（包括寄存器的值和程序计数器等），并切换到另一个线程继续执行。当被暂停的线程所需的资源可用时，处理器会恢复该线程的上下文并继续执行。
- 指令级并行：除了线程切换外，硬件多线程技术还可以利用指令级并行（Instruction-Level Parallelism，简称ILP）来提高处理器的效率。指令级并行允许处理器在同一时间内执行多条指令，这些指令可能来自同一个线程或多个线程。通过利用指令级并行，处理器可以更有效地利用其执行单元和流水线，从而提高整体的执行效率。

六、硬件多线程的基本概念

4、硬件多线程的优势

- 提高处理器利用率：硬件多线程技术能够充分利用处理器的执行单元和流水线，即使在处理那些包含大量等待时间的线程时也能保持较高的利用率。
- 提高系统吞吐量：通过同时运行多个线程，硬件多线程技术可以显著提高系统的吞吐量，使得处理器能够在更短的时间内完成更多的任务。
- 增强系统响应性：硬件多线程技术能够使得处理器在处理多个任务时更加灵活和高效，从而增强系统的响应性和用户体验。例如，在操作系统中，硬件多线程技术可以使得系统能够更快地响应用户的请求和中断，提高系统的整体性能和稳定性。

单选题 1分

设置

【2022统考真题】下列关于并行处理技术的叙述中，不正确的是(　　)。

A 多核处理器属于MIMD结构

B 向量处理器属于SIMD结构

C 硬件多线程技术只可用于多核处理器

D SMP中所有处理器共享单一物理地址空间

提交